# INSTITUTE NAME: CSPIT
# FACULTY OF ENGINEERING AND TECHNOLOGY
## BRANCH: IT
## OBJECT ORIENTED PROGRAMMING (CEUE203)
## A.Y. 2026–27 | SEMESTER – 3

# PRACTICAL INDEX

**Name: Kavya Bhadiyadra**
**Class: IT-1**
**ID No.: 25IT006**


| Sr. No. | Date | Practical Name | Page No. |
|---|---|---|---|
| 1 | 12/07/2026 | Vending Machine, Toll Booth and Rock-Paper-Scissors-Lizard-Spock | 2 |
| 2 | 19/07/2026 | Smart Thermostat, Cinema Show and Parking Lot | 7 |
| 3 | 26/07/2026 | Distinct Points, Duplicate Card Check and Fractions | 20 |
| 4 | 02/08/2026 | Password Strength, Chat-Log Filter and Template Filler | 38 |
| 5 | 23/08/2026 | Shape Areas, Payroll and Media Late-Fee Calculation | 55 |
| 6 | 31/10/2026 | Remote Control, Notification Senders and Discount Engine | 60 |
| 7 | 29/10/2026 | Form Validator, Mini Test Runner and Column Annotation | 67 |
| 8 | 23/09/2026 | Guarded Calculator, Stock Issue and Auto Closeable Resource | 75 |
| 9 | 23/09/2026 | Counter Race, Seat Booking Race and Parallel Array Summation | 85 |
| 10 | 11/10/2026 | Thread-Pool Runner, Producer–Consumer and Deadlock | 93 |
| 11 | 29/10/2026 | Save/Load, Log Analyzer and NIO Folder Report | 100 |
| 12 | 29/10/2026 | Word Frequency Using HashMap and Comparator | 110 |
| 13 | 29/10/2026 | Generic Containers, Bounded Statistics and Generic Stack | 116 |
| 14 | 29/10/2026 | Layered Application Assembly and JUnit Testing | 124 |

# PRACTICAL 1

## Part A

1. Vending machine (VendingMachine.java, main in this file)

(a) Define an enum Coin with constants ONE, TWO, FIVE, TEN.

(b) In main, set a snack price of 15 and a running total of 0; create a Scanner.

(c) Loop: read a coin name, use a switch expression to convert the Coin to its value

(ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so

far.

(d) Stop the loop once the total reaches 15 or more.

(e) Print the change to return (total − 15).

# CODE:

## 25IT006 KAVYA BHADIYADRA 2 CEUE203–Object Oriented Programming

```
import java.util.Scanner;

public class VendingMachine {

 enum Coin {
 ONE, TWO, FIVE, TEN
 }

 public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

 int price = 15;
 int total = 0;

 while (total < price) {

 System.out.print("Enter coin (ONE, TWO, FIVE, TEN): ");  String
input = sc.next().toUpperCase();

 Coin coin = Coin.valueOf(input);

 int value = switch (coin) {
```

```
case ONE -> 1;
case TWO -> 2;
case FIVE -> 5;
case TEN -> 10;
```

**25IT006 KAVYA BHADIYADRA 3 CEUE203–Object Oriented Programming**

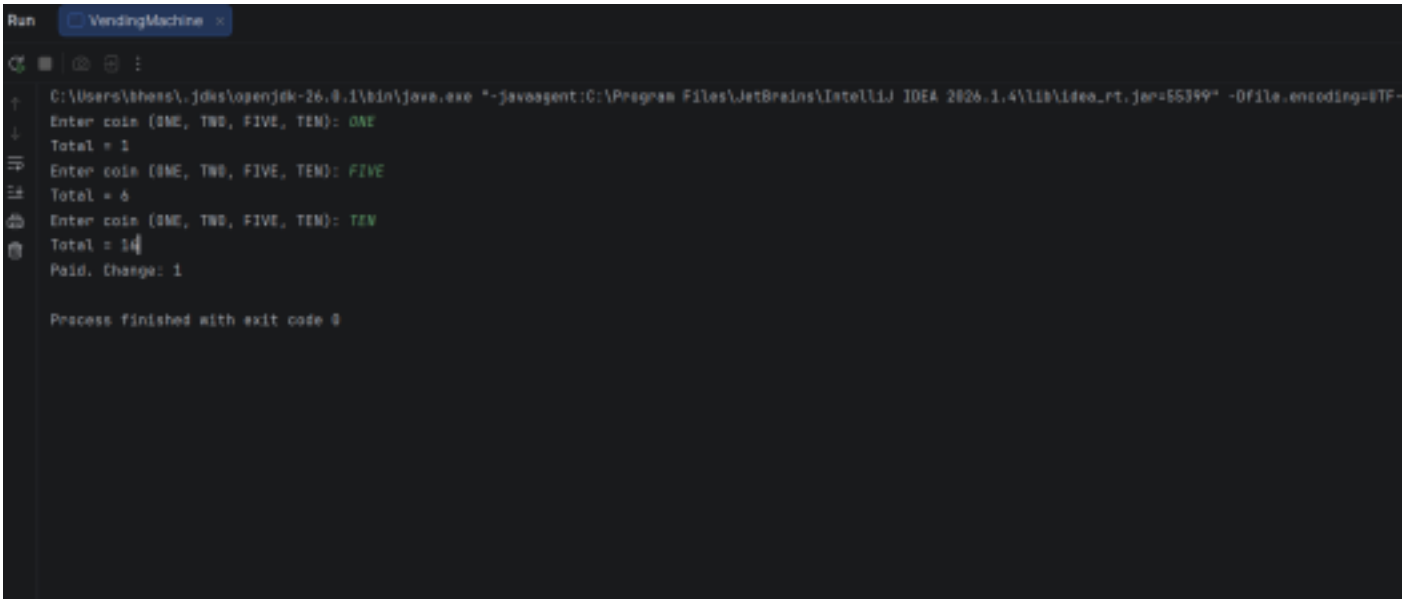
```
};

total += value;
System.out.println("Total = " + total);
}

System.out.println("Paid. Change: " + (total - price));

sc.close();
}
}
```

# OUTPUT:




**25IT006 KAVYA BHADIYADRA 4 CEUE203–Object Oriented Programming**

**B : Toll booth (TollBooth.java, main in this file)**
**(a) Define a record Vehicle(String number, String type).**
**(b) In main, keep a running toll total and three counters (bike, car, truck).**
**(c) Loop until the user types “done” for the number: build a Vehicle, use a switch**
**expression on its type for the toll (bike→20, car→50, truck→150), add to the total,**
**and increment that type’s counter.**
**(d) After the loop, print the total toll and the type with the highest count.**
**Expected output: two cars + one bike → “Total toll: 120”, “Most frequent: car”.**

**3. (For Advanced Learners – home)**
**Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)**
**(a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK.**
**(b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie —**
**use a switch expression listing what each move beats (Scissors cut Paper, Paper**
**covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors,**
**and the five mirror cases).**
**(c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call**
**winner(), print the round result and keep score.**
**(d) After 5 rounds, print the overall winner.**
**Expected output: each round prints both moves and the round winner; ends with a**
**"You win 3–2" summary.**

**25IT006 KAVYA BHADIYADRA 1 CEUE203–Object Oriented Programming**

# PRACTICAL 2

## Part A

### 1.mart thermostat (Thermostat.java, main in this file)

(a)

Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0.

(b)

Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount.

(c)

Constructor(location): chain with this(location, 22).

(d)

raise(): if temperature < MAX add 1, else print “Already at maximum (30)”.

(e)

lower(): if temperature > MIN subtract 1, else print “Already at minimum (16)”.

(f)Add getTemperature() (no setter) and static getActiveCount().

(g) In

main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount().

**Expected output:** *temperature rises to 30 then "Already at maximum",*

# CODE:

```
public class Thermostat {

    private String location;
    private int temperature;

    private static final int MIN = 16;
    private static final int MAX = 30;

    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;

        if (startTemp >= MIN && startTemp <= MAX) {
            this.temperature = startTemp;
        } else {
            this.temperature = 22;
        }

        activeCount++;
    }


    public Thermostat(String location) {
        this.location = location;
        this.temperature = 22;
    }
```

```
    public void raise() {
        if (temperature < MAX) {
            temperature++;
        } else {
            System.out.println("Already at maximum (30)");
        }
    }


    public void lower() {
        if (temperature > MIN) {
            temperature--;
        } else {
            System.out.println("Already at minimum (16)");
        }
    }


    public int getTemperature() {
        return temperature;
    }


    public static int getActiveCount() {
        return activeCount;
    }

    public static void main(String[] args) {

        Thermostat t1 = new Thermostat("Living Room", 25);
        Thermostat t2 = new Thermostat("Bedroom");

        System.out.println("Raising temperature:");

        for (int i = 0; i < 10; i++) {
            t1.raise();
            System.out.println("Temperature = " + t1.getTemperature());
```

```
        }

        System.out.println("\nLowering temperature:");

        for (int i = 0; i < 20; i++) {
            t1.lower();
            System.out.println("Temperature = " + t1.getTemperature());
        }

        System.out.println("\nActive Thermostats = " +
Thermostat.getActiveCount());
    }
}
```

# OUTPUT:

```
c:\Users\USER\OneDrive\Attachments\Desktop\GitHub\myfirstgithub\java>cd "c:\Users\USER\OneDrive\Attachments\Desktop\GitHu
 javac Thermostat.java && java Thermostat
Raising temperature:
Temperature = 26
Temperature = 27
Temperature = 28
Temperature = 29
Temperature = 30
Already at maximum (30)
Temperature = 30
Already at maximum (30)
Temperature = 30
Already at maximum (30)
Temperature = 30
Already at maximum (30)
Temperature = 30
Already at maximum (30)
Temperature = 30

Lowering temperature:
Temperature = 29
Temperature = 28
Temperature = 27
Temperature = 26
Temperature = 25
Temperature = 24
Temperature = 23
Temperature = 22
Temperature = 21
Temperature = 20
Temperature = 19
Temperature = 18
Temperature = 17
Temperature = 16
Already at minimum (16)
```

```
Lowering temperature:
Temperature = 29
Temperature = 28
Temperature = 27
Temperature = 26
Temperature = 25
Temperature = 24
Temperature = 23
Temperature = 22
Temperature = 21
Temperature = 20
Temperature = 19
Temperature = 18
Temperature = 17
Temperature = 16
Already at minimum (16)
Temperature = 16
Already at minimum (16)
Temperature = 16
Already at minimum (16)
Temperature = 16
Already at minimum (16)
Temperature = 16
Already at minimum (16)
Temperature = 16
Already at minimum (16)
Temperature = 16

Active Thermostats = 1

c:\Users\USER\OneDrive\Attachments\Desktop\GitHub\myfirstgithub\java>
```

## 25IT006 KAVYA BHADIYADRA 4 CEUE203–Object Oriented Programming

## 2.Cinema show (CinemaShow.java, main in this file)

(a)

Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0.

(b)

Constructor(title, capacity): set both and seatsAvailable = capacity.
Constructor(title): chain with this(title, 100).

(c)

book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged.

(d)
cancel(int n): increase seatsAvailable by n but never above capacity.
(e)
Add getSeatsAvailable() and static getTotalBooked().
(f) In
main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked().
**Expected output:** *booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.*

```
public class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    // Constructor with title and capacity
    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        this.seatsAvailable = capacity;
    }

    // Constructor with default capacity
    public CinemaShow(String title) {
        this(title, 100);
    }

    // Book seats
    public boolean book(int n) {
        if (n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }
```

```
    // Cancel seats
    public void cancel(int n) {
        seatsAvailable += n;
        if (seatsAvailable > capacity) {
            seatsAvailable = capacity;
        }
    }

    // Getter for available seats
    public int getSeatsAvailable() {
        return seatsAvailable;
    }

    // Static getter for total booked seats
    public static int getTotalBooked() {
        return totalBooked;
    }

    // Main method
    public static void main(String[] args) {
        CinemaShow show = new CinemaShow("Avengers", 50);

        System.out.println("Book 20: " + show.book(20));
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

        System.out.println("Book 25: " + show.book(25));
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

        System.out.println("Book 10: " + show.book(10)); // Not enough
seats
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

        show.cancel(15);
        System.out.println("After cancel 15, Seats Available: " +
```

```
show.getSeatsAvailable());

        System.out.println("Book 10: " + show.book(10));
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

        show.cancel(50); // Cannot exceed capacity
        System.out.println("After cancel 50, Seats Available: " +
show.getSeatsAvailable());

        System.out.println("Total Booked Seats: " +
CinemaShow.getTotalBooked());
    }
}
```

# Output:

```
c:\Users\USER\OneDrive\Attachments\Desktop\GitHub\myfirstgithub\java>cd "c:\Users\USER\OneDrive\Attachments\Desktop\GitHub\myfirstgithub\java\" &&
 javac CinemaShow.java && java CinemaShow
Book 20: true
Seats Available: 30
Book 25: true
Seats Available: 5
Book 10: false
Seats Available: 5
After cancel 15, Seats Available: 20
Book 10: true
Seats Available: 10
After cancel 50, Seats Available: 50
Total Booked Seats: 55
```

## 3.(For Advanced Learners – home) Parking lot (ParkingLot.java)

(a)
Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0.

(b)
park(String type): for “two”, if twoWheelers < twoCap increment and add 20

to revenue; for “four”, if fourWheelers < fourCap increment and add 40; otherwise print “Full” and reject.

(c)
leave(String type): decrement the matching count but never below 0.

(d) In
main: simulate a sequence of park/leave events, printing each outcome, then
print final occupancy and revenue.

**Expected output:** *parking into a full section is refused; revenue = 20×(two parked) + 40×(four parked).*

```
public class ParkingLot {
    private int twoWheelers;
    private int fourWheelers;
    private final int twoCap;
    private final int fourCap;
    private static long revenue = 0;

    // Constructor
    public ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
        this.twoWheelers = 0;
        this.fourWheelers = 0;
    }

    // Park vehicle
    public void park(String type) {
        if (type.equalsIgnoreCase("two")) {
            if (twoWheelers < twoCap) {
                twoWheelers++;
                revenue += 20;
                System.out.println("Two-wheeler parked.");
```

```
            } else {
                System.out.println("Two-wheeler section Full");
            }
        } else if (type.equalsIgnoreCase("four")) {
            if (fourWheelers < fourCap) {
                fourWheelers++;
                revenue += 40;
                System.out.println("Four-wheeler parked.");
            } else {
                System.out.println("Four-wheeler section Full");
            }
        } else {
            System.out.println("Invalid vehicle type.");
        }
    }

    // Vehicle leaves
    public void leave(String type) {
        if (type.equalsIgnoreCase("two")) {
            if (twoWheelers > 0) {
                twoWheelers--;
                System.out.println("Two-wheeler left.");
            } else {
                System.out.println("No two-wheeler to leave.");
            }
        } else if (type.equalsIgnoreCase("four")) {
            if (fourWheelers > 0) {
                fourWheelers--;
                System.out.println("Four-wheeler left.");
            } else {
                System.out.println("No four-wheeler to leave.");
            }
        } else {
            System.out.println("Invalid vehicle type.");
        }
    }

    // Getters
```

```
    public int getTwoWheelers() {
        return twoWheelers;
    }

    public int getFourWheelers() {
        return fourWheelers;
    }

    public static long getRevenue() {
        return revenue;
    }

    // Main method
    public static void main(String[] args) {
        ParkingLot lot = new ParkingLot(2, 2);

        lot.park("two");
        lot.park("two");
        lot.park("two");      // Full

        lot.park("four");
        lot.park("four");
        lot.park("four");     // Full

        lot.leave("two");
        lot.leave("four");

        lot.park("two");
        lot.park("four");

        System.out.println("\nFinal Occupancy:");
        System.out.println("Two-wheelers: " + lot.getTwoWheelers());
        System.out.println("Four-wheelers: " + lot.getFourWheelers());

        System.out.println("Revenue: " + ParkingLot.getRevenue());
    }
}
```

# Output:

```
c:\Users\USER\OneDrive\Attachments\Desktop\GitHub\myfirstgithub\java>java ParkingLot
Two-wheeler parked.
Two-wheeler parked.
Two-wheeler section Full
Four-wheeler parked.
Four-wheeler parked.
Four-wheeler section Full
Two-wheeler left.
Four-wheeler left.
Two-wheeler parked.
Four-wheeler parked.

Final Occupancy:
Two-wheelers: 2
Four-wheelers: 2
Revenue: 180
```

# Practical 3

## 1. Distinct points (Point.java + Driver.java with main)

(a) Create class Point with private int x, y and a constructor.

(b) Override toString() to return “(x, y)”.

(c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)).

(d) In Driver.main: build a Point[] with some repeated coordinates.

(e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not.

(f) Print “Distinct: N”.

## CODE:

## Point.java

```
import java.util.Objects;

public class Point {

    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    @Override
    public boolean equals(Object obj) {
```

```
        Point p = (Point) obj;

        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

## Driver.java

```
import java.util.Scanner;

public class Driver {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of points: ");
        int n = sc.nextInt();

        Point[] points = new Point[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter x and y for Point " + (i + 1) + ": ");
            int x = sc.nextInt();
            int y = sc.nextInt();

            points[i] = new Point(x, y);
        }

        int count = 0;

        for (int i = 0; i < points.length; i++) {

            boolean found = false;

            for (int j = 0; j < i; j++) {

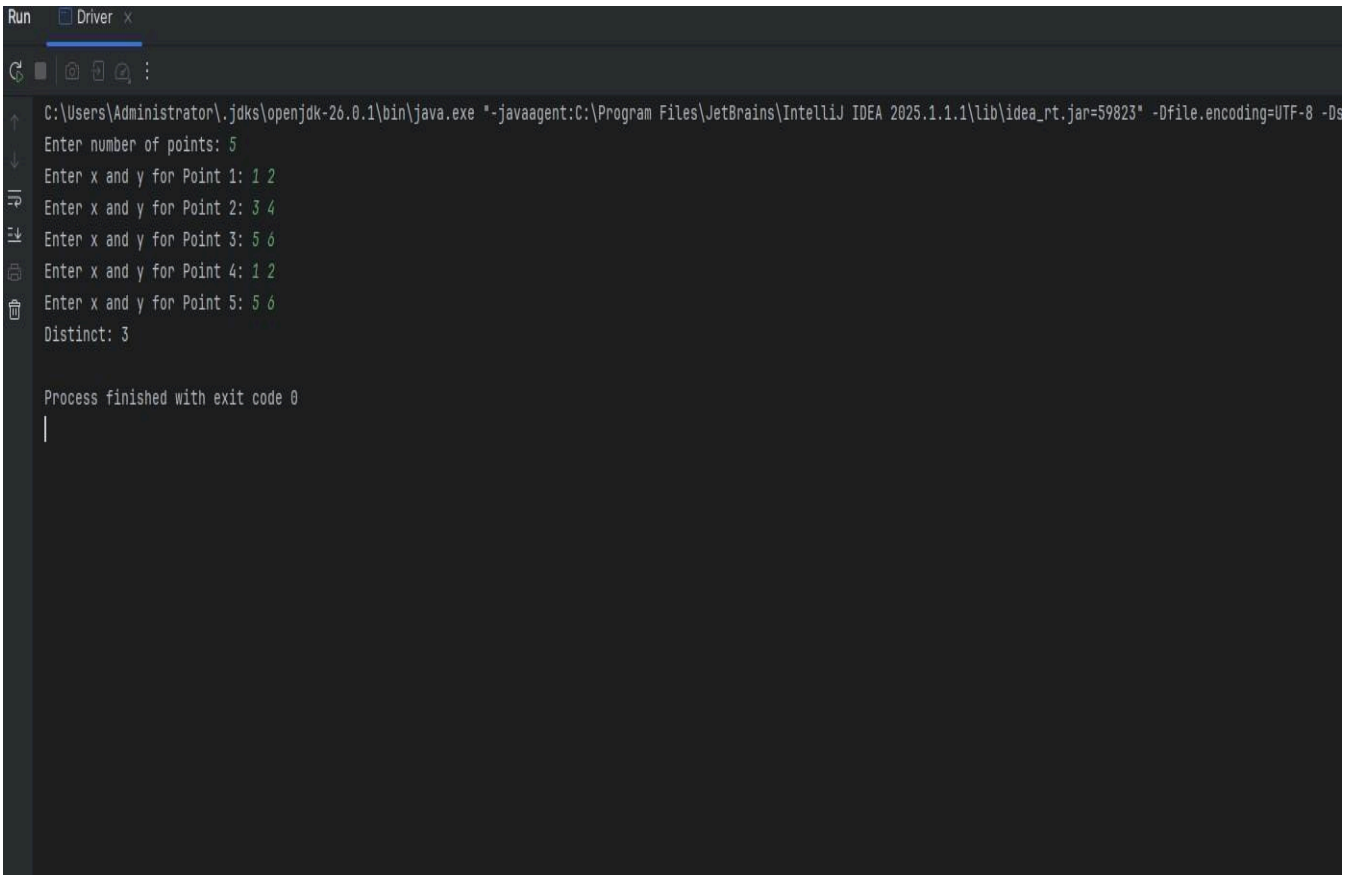
                if (points[i].equals(points[j])) {
                    found = true;
                    break;
                }
            }

            if (!found) {
                count++;
            }
        }
```

```
        System.out.println("Distinct: " + count);

        sc.close();
    }
}
```

# output




## 2. Duplicate card check (Card.java + Driver.java)

(a) Create class Card with private String rank, suit and a constructor.

(b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts").

(c) Override equals()/hashCode() by rank + suit.

(d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals()to check it against the earlier cards.

(e) Print “Duplicate found: <card>” the first time a repeat appears.

## CODE:

## Card.java

```
import java.util.Objects;

public class Card {

   private String rank;
   private String suit;

   public Card(String rank, String suit) {
       this.rank = rank;
       this.suit = suit;
   }

   @Override
   public String toString() {
       return rank + " of " + suit;
   }

   @Override
   public boolean equals(Object obj) {
```

```
        if (this == obj)
            return true;

        if (!(obj instanceof Card))
            return false;

        Card c = (Card) obj;

        return rank.equals(c.rank) && suit.equals(c.suit);
    }

    @Override
    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

# Driver.java

```
import java.util.Scanner;

public class Driver {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of cards: ");
        int n = sc.nextInt();
        sc.nextLine();
```

```
        Card[] cards = new Card[n];

        boolean duplicate = false;

        for (int i = 0; i < n; i++) {

            System.out.print("Enter Rank: ");
            String rank = sc.nextLine();

            System.out.print("Enter Suit: ");
            String suit = sc.nextLine();

            Card newCard = new Card(rank, suit);

            for (int j = 0; j < i; j++) {
                 if (newCard.equals(cards[j]))
{
                    System.out.println("Duplicate found: " + newCard);
                    duplicate = true;
                    break;
                }
            }

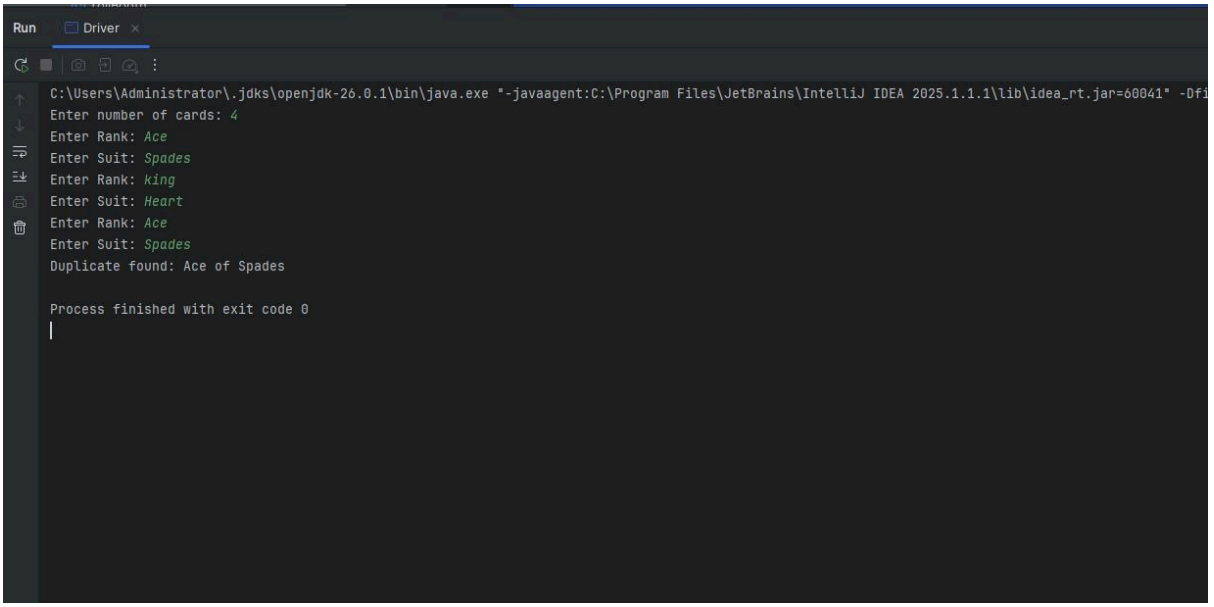
            cards[i] = newCard;

            if (duplicate)
                break;
        }

sc.close();

    }
}
```

# OUTPUT




## 3. Fractions (Fraction.java + Driver.java)

(a) Create class Fraction with private int num, den.

(b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g.

(c) Override toString() to return num + "/" + den.

(d) Override equals()/hashCode() using the reduced num and den.

(e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.

# CODE:

# Fraction.java

```
public class Fraction {


   private int num;
private int den;



   public Fraction(int num, int den) {

       int g = gcd(num, den);
this.num = num / g;


       this.den = den / g;

   }


   private int gcd(int a, int b) {
while (b != 0) {


           int temp = b;

           b = a % b;

           a = temp;

       }

       return a;

   }

    public String toString()
{


       return num + "/" + den;

   }


   public boolean equals(Object obj) {
Fraction f = (Fraction) obj;


       return num == f.num && den == f.den;

   }
```

```
    public int hashCode()
{

        return num * 31 + den;

    }

}
```

# Driver.java

```
import java.util.Scanner;

public class Driver {

    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);



        System.out.print("Enter numerator of Fraction 1: ");

        int n1 = sc.nextInt();

        System.out.print("Enter denominator of Fraction 1: ");
int d1 = sc.nextInt();



        System.out.print("Enter numerator of Fraction 2: ");

        int n2 = sc.nextInt();

        System.out.print("Enter denominator of Fraction 2: ");

        int d2 = sc.nextInt();


        Fraction f1 = new Fraction(n1, d1);

        Fraction f2 = new Fraction(n2, d2);


        System.out.println("\nFraction 1 = " + f1);

        System.out.println("Fraction 2 = " + f2);

         if
(f1.equals(f2))

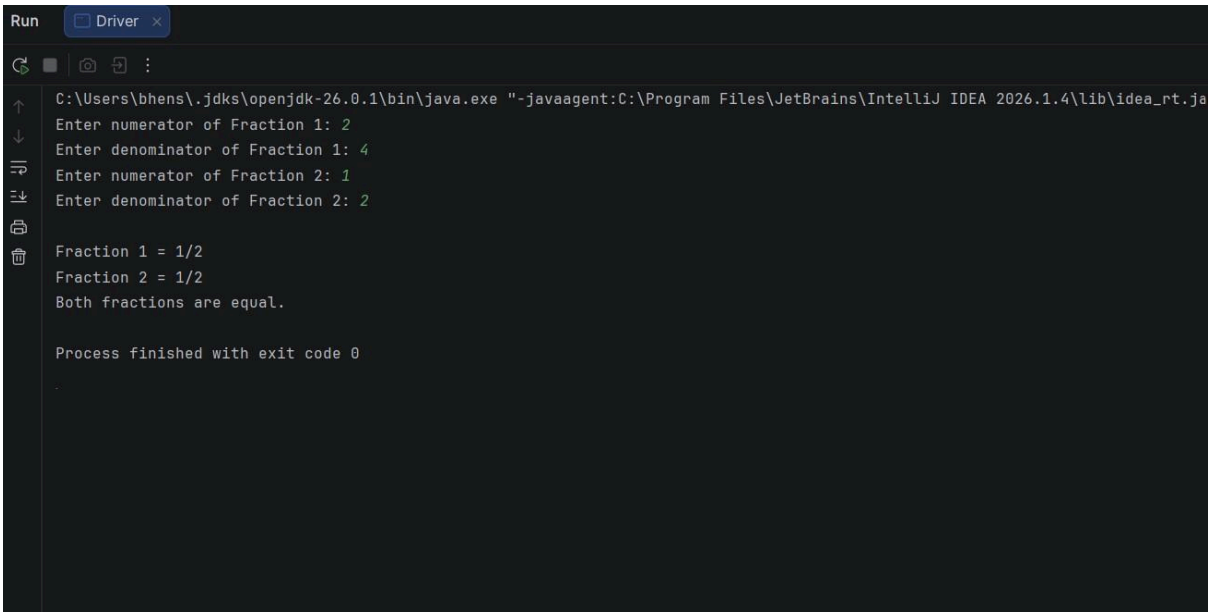
            System.out.println("Both fractions are equal.");
```

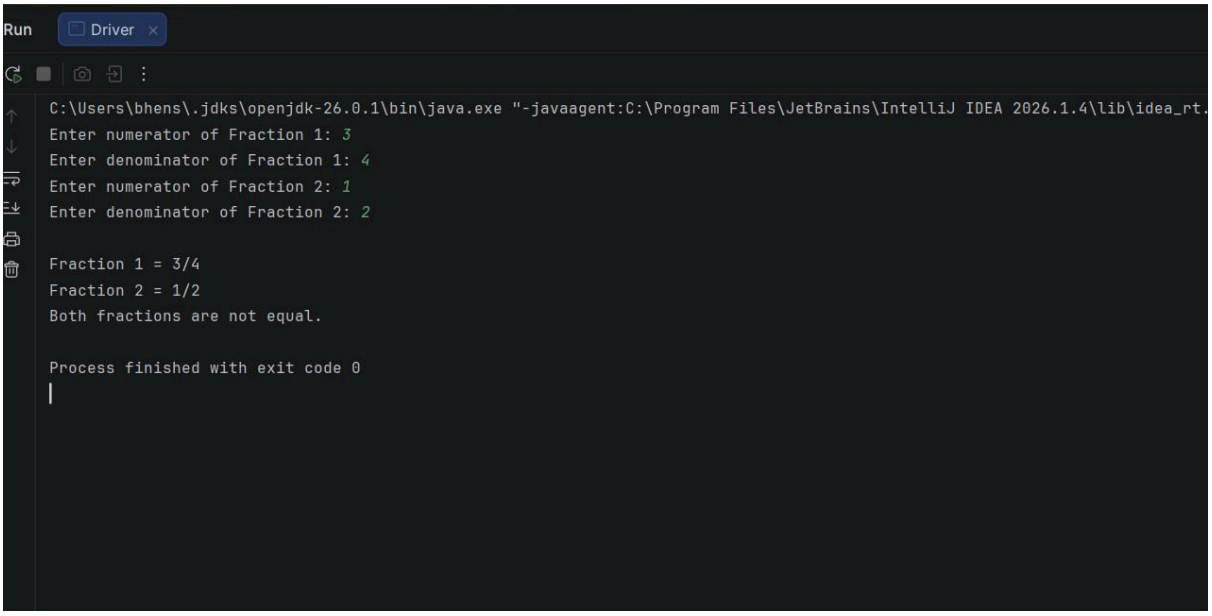
```
        else
            System.out.println("Both fractions are not equal.");

sc.close();

    }
}
```

# OUTPUT



# Part B —Project Work: MiniBank

1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance.

2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal.

3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method.

4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer.

5. In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type.

## Program 1: Customer.java

```
public class Customer implements Cloneable {


   private String name;
```

```
   private Address address;


   public Customer(String name, Address address) {
       this.name = name;
       this.address = address;
   }


   public Address getAddress() {
       return address;
   }
    public Customer clone()
{

       return new Customer(name, address);
   }
    public static class Address
{



       private String line;
       private String city;
       private String pincode;


       public Address(String line, String city, String pincode) {
           this.line = line;
           this.city = city;
this.pincode = pincode;

       }


       public String getLine() {
return line;

       }


       public String getCity() {
           return city;
       }
```

```
        public String getPincode() {
            return pincode;
        }
    }
}
```

# Program 2: Account.java

```
public class Account {


    private int accountNumber;
    private String ownerName;
    private double balance;


    public Account(int accountNumber, String ownerName, double balance) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
this.balance = balance;


    }


    public String toString() {         return "Account No: " + accountNumber
+ ", Name: " + ownerName + ",
Balance: " + balance;
    }


    public boolean equals(Object obj) {
        Account a = (Account) obj;
        return this.accountNumber == a.accountNumber;
    }
     public int hashCode()
{


        return accountNumber;
    }
```

```
}
```

## Program 3: Driver.java

```
import java.util.Scanner;


public class Driver {


```

```
   public static void main(String[] args) {


       Scanner sc = new Scanner(System.in);


       System.out.print("How many accounts? ");
       int n = sc.nextInt();
       sc.nextLine();


       Account[] accounts = new Account[n];


       for (int i = 0; i < n; i++) {
           System.out.println("\nEnter Details of Account " + (i + 1));


           System.out.print("Account Number: ");
           int accNo = sc.nextInt();
           sc.nextLine();


           System.out.print("Owner Name: ");
           String name = sc.nextLine();


           System.out.print("Balance: ");
           double balance = sc.nextDouble();
           sc.nextLine();
```

```
            accounts[i] = new Account(accNo, name, balance);
        }


        System.out.println("\nAccount Details");
         for (int i = 0; i < n; i++)
{
            System.out.println(accounts[i]);
        }


        System.out.println();


        if (accounts[0].equals(accounts[1]))
                    System.out.println("Account 1 and Account 2 are Equal");
else
            System.out.println("Account 1 and Account 2 are Not Equal");


        if (accounts[0] instanceof Account)
            System.out.println("accounts[0] is an Account object");


        sc.close();
    }
}
```

# OUTPUT




# KEY QUESTION:

## 1. Why must equals() and hashCode() be overridden together?

**ANS:** equals() and hashCode() should always be overridden together because they work as a pair. The equals() method checks whether two objects are logically equal, while the hashCode() method returns a hash value for an object. If two objects are considered equal by equals(), they must return the same hash code. Otherwise, collections such as HashSet, HashMap, and Hashtable may not work correctly and can store duplicate objects or fail to find existing ones. Therefore, overriding both methods together ensures correct object comparison, proper storage, and efficient searching in Java collections.

## 2. What does the default Object.toString() return, and why override it?

**ANS:** The default Object.toString() method returns a string containing the object's class name followed by its hash code in hexadecimal format, such as Account@1a2b3c. This output is not easy for users to understand. Therefore, we override toString() to return meaningful and readable information about the object, such as its account number, owner name, and balance. This makes the program output more user-friendly and easier to debug.

## 3. What is the difference between a static nested class and an inner class?

**ANS:** A static nested class is a class declared with the static keyword inside another class. It can be created without creating an object of the outer class and can access only the static members of the outer class directly. An inner class (non-static nested class) is declared without the static keyword and requires an object of the outer class to be created. It can access both the instance and static members of the outer class. Therefore, a static nested class is independent of the outer class object, while an inner class depends on it.

## LEARNING OUTCOME:

**CEUE203–Object Oriented Programming 25IT006–Kavya**

After completing this practical, students will be able to create Java classes by overriding the toString(), equals(), and hashCode() methods for meaningful object representation and correct object comparison. Students will also learn to use nested classes, implement object cloning with Cloneable, use the instanceof operator, and apply these concepts to build reliable object-oriented applications such as the MiniBank project.

# Practical 4 :

## 1. Password strength (PasswordChecker.java + Driver.java)

(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.

(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4).

(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.

## CODE:

## PasswordChecker.java

```
public class PasswordChecker {

public boolean checkLength(String pw) {
return pw.length() >= 8;
}

public boolean checkUppercase(String pw) {
for (int i = 0; i < pw.length(); i++) {
if (Character.isUpperCase(pw.charAt(i))) {
return true;
```

```
}
}
return false;
}

public boolean checkDigit(String pw) {
for (int i = 0; i < pw.length(); i++) {
if (Character.isDigit(pw.charAt(i))) {
return true;
}
}
return false;
}

public boolean checkSpecial(String pw) {
for (int i = 0; i < pw.length(); i++) {
char ch = pw.charAt(i);
if (!Character.isLetterOrDigit(ch)) {
return true;
}
}
return false;
}

public String strength(String pw) {
int count = 0;

if (checkLength(pw))
count++;
if (checkUppercase(pw))
count++;
if (checkDigit(pw))
count++;
if (checkSpecial(pw))

count++;

if (count <= 1)
```

```
return "Weak";
else if (count <= 3)
return "Medium";
else
return "Strong";
}
```

# Driver.java

```
import java.util.Scanner;

public class Driver {
public static void main(String[] args) {

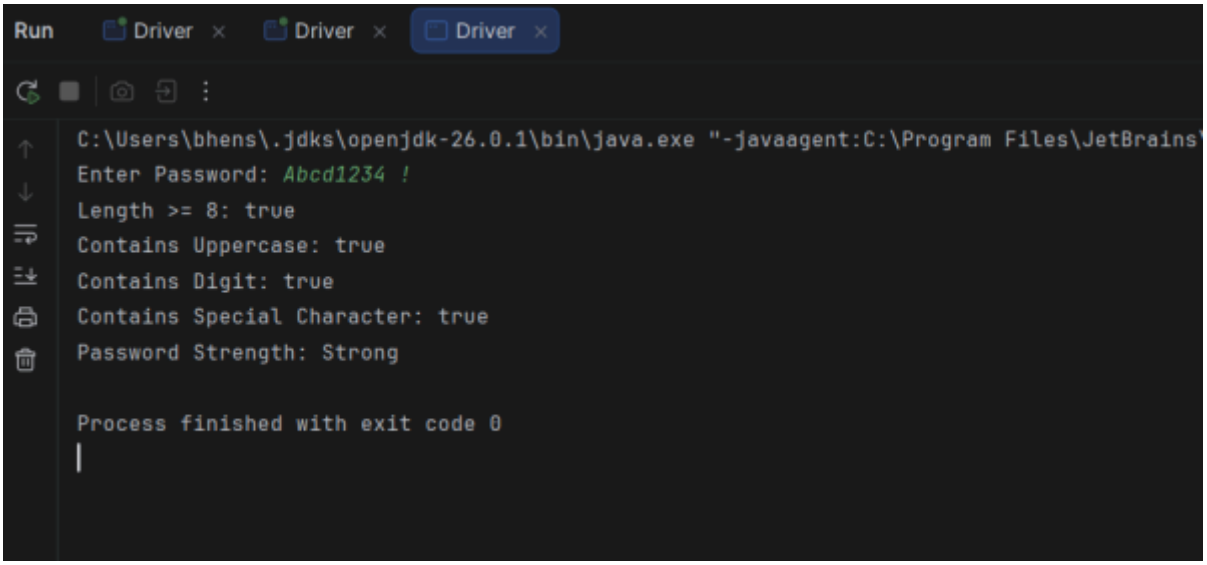
Scanner sc = new Scanner(System.in);
PasswordChecker pc = new PasswordChecker();

System.out.print("Enter Password: ");
String password = sc.nextLine();


System.out.println("Length >= 8: " + pc.checkLength(password));
System.out.println("Contains Uppercase: " + pc.checkUppercase(password));
System.out.println("Contains Digit: " + pc.checkDigit(password));
System.out.println("Contains Special Character: " +
pc.checkSpecial(password));
System.out.println("Password Strength: " + pc.strength(password));

sc.close();
}
```

# OUTPUT:




## 2. Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like “10:05 alice

Hello there” (include one malformed line with fewer than 3 parts).

(b) Read a keyword from the user.

(c) For each line, split into time/user/message with split(“ ”, 3); skip

lines that do not have 3 parts.

(d) If the message contains the keyword (case-insensitive),

increment a counter and append “time user: message” to a

StringBuilder.

(e) Print “Matches: N” and the built report.

# ChatFilter.java

```
public class ChatFilter {

public String filter(String[] logs, String keyword) {

StringBuilder report = new StringBuilder();
int count = 0;

for (int i = 0; i < logs.length; i++) {

String[] parts = logs[i].split(" ", 3);

if (parts.length < 3) {
continue;
}

String time = parts[0];
String user = parts[1];
String message = parts[2];


if (message.toLowerCase().contains(keyword.toLowerCase())) {
count++;
report.append(time)
.append(" ")
.append(user)
.append(": ")
.append(message)
.append("\n");

}
}

return "Matches: " + count + "\n" + report.toString();
}
}
```

# Driver.java

```
import java.util.Scanner;

public class Driver {
```

```
public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

String[] logs = {
"10:05 alice Hello there",
"10:10 bob Good Morning",
"WrongLine",
"10:20 charlie Hello Everyone"
};

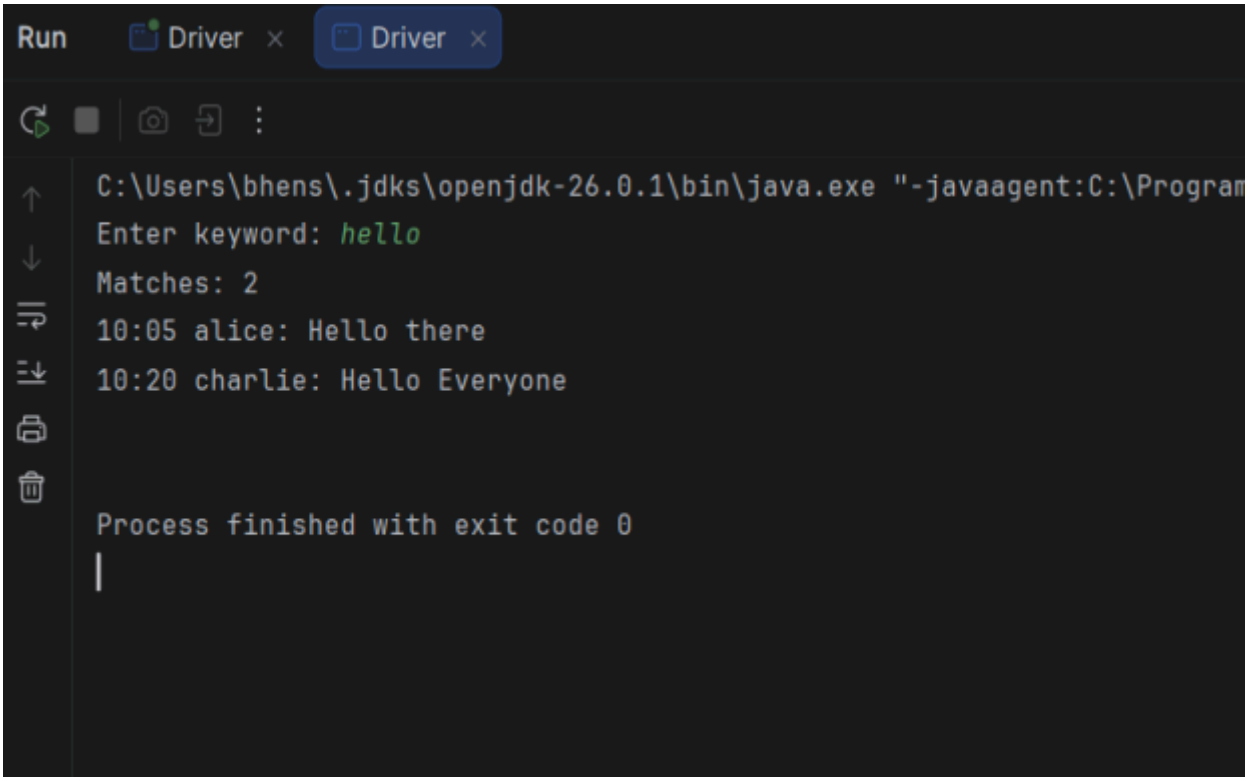
System.out.print("Enter keyword: ");
String keyword = sc.nextLine();

ChatFilter cf = new ChatFilter();

System.out.println(cf.filter(logs, keyword));

sc.close();
}
}
```

## OUTPUT




## 3. Template filler (TemplateFiller.java + Driver.java)

(a) In Driver.main, define a template like “Dear {name}, order

{id} ships {date}.” and two parallel arrays: names {“name”,“id”}

and values {“Riya”, “A07”} (note {date} is not supplied).

(b) Compile a Pattern for \{(\w+)\} and get a Matcher on the

(c) Walk the matches with a StringBuilder: for each

{placeholder}, look up its value in the names array and

replace with the value, or with “[?]” if not found.

(d) Print the filled template.
CODE:

## TemplateFiller.java

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class TemplateFiller {

public String fillTemplate(String template, String[] names, String[]
values) {

Pattern pattern = Pattern.compile("\\{(\\w+)\\}");

Matcher matcher = pattern.matcher(template);

StringBuilder result = new StringBuilder();

while (matcher.find()) {

String key = matcher.group(1);
String value = "[?]";

for (int i = 0; i < names.length; i++) {
if (key.equals(names[i])) {
value = values[i];
break;
}
}

matcher.appendReplacement(result, value);
}
matcher.appendTail(result);
return result.toString();
}
}
```

## Driver.java

```
public class Driver {

public static void main(String[] args) {

String template = "Dear {name}, order {id} ships {date}.";

String[] names = {"name", "id"};
String[] values = {"Riya", "A07"};

TemplateFiller tf = new TemplateFiller();

System.out.println(tf.fillTemplate(template, names, values));
}
```

## OUTPUT




## Part B —Project Work:

## MiniBank

1. Create a class named Validator with public static methods

that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean.
2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount.
3. Create a class named CommandParser with a public static method parse(String line) that splits a line such as “DEPOSIT AC0001 500” with split(), converts the parts, and returns a Command object.
4. Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string.
5. In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts.

# Validator.java

```
import java.util.regex.Pattern;

public class Validator {

static Pattern mobile = Pattern.compile("^[6-9][0-9]{9}$");

static Pattern email = Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");

static Pattern pan = Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

static Pattern ifsc = Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

public static boolean isValidMobile(String s) {

return mobile.matcher(s).matches();
```

```
}

public static boolean isValidEmail(String s) {

return email.matcher(s).matches();

}


public static boolean isValidPan(String s) {

return pan.matcher(s).matches();

}

public static boolean isValidIfsc(String s) {

return ifsc.matcher(s).matches();

}

}
```

## TransactionType.java

```
public enum TransactionType {

DEPOSIT,

WITHDRAW,

TRANSFER
```

## Command.java

```
public record Command(TransactionType type, String accountNumber, long amount) {
```

```
}
```

## CommandParser.java

```
public class CommandParser {

public static Command parse(String line) {

String[] parts = line.split(" ");

TransactionType type = TransactionType.valueOf(parts[0]);

String accountNumber = parts[1];

long amount = Long.parseLong(parts[2]);

return new Command(type, accountNumber, amount);

}
```

## Account.java

```
public class Account {

String accountNumber;

String name;

long balance;

public Account(String accountNumber, String name, long balance) {

this.accountNumber = accountNumber;

this.name = name;
```

```
this.balance = balance;

}

}
```

## StatementFormatter.java

```
public class StatementFormatter {

public static String buildStatement(Account account) {

StringBuilder sb = new StringBuilder();

sb.append("Account Number: ").append(account.accountNumber).append("\n");

sb.append("Name: ").append(account.name).append("\n");

sb.append("Balance: ").append(account.balance);

return sb.toString();

}

}
```

## Driver.java

```
import java.util.Scanner;

public class Driver {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter Mobile: ");

String mobile = sc.nextLine();

System.out.print("Enter Email: ");

String email = sc.nextLine();

System.out.print("Enter PAN: ");

String pan = sc.nextLine();

System.out.print("Enter IFSC: ");

String ifsc = sc.nextLine();

System.out.println("Mobile Valid: " + Validator.isValidMobile(mobile));

System.out.println("Email Valid: " + Validator.isValidEmail(email));

System.out.println("PAN Valid: " + Validator.isValidPan(pan));

System.out.println("IFSC Valid: " + Validator.isValidIfsc(ifsc));

System.out.print("Enter Command (Example: DEPOSIT AC0001 500): ");

String line = sc.nextLine();

Command cmd = CommandParser.parse(line);

System.out.println("Type: " + cmd.type());

System.out.println("Account: " + cmd.accountNumber());

System.out.println("Amount: " + cmd.amount());

Account acc = new Account(cmd.accountNumber(), "Riya", 10000);
```

```
System.out.println();

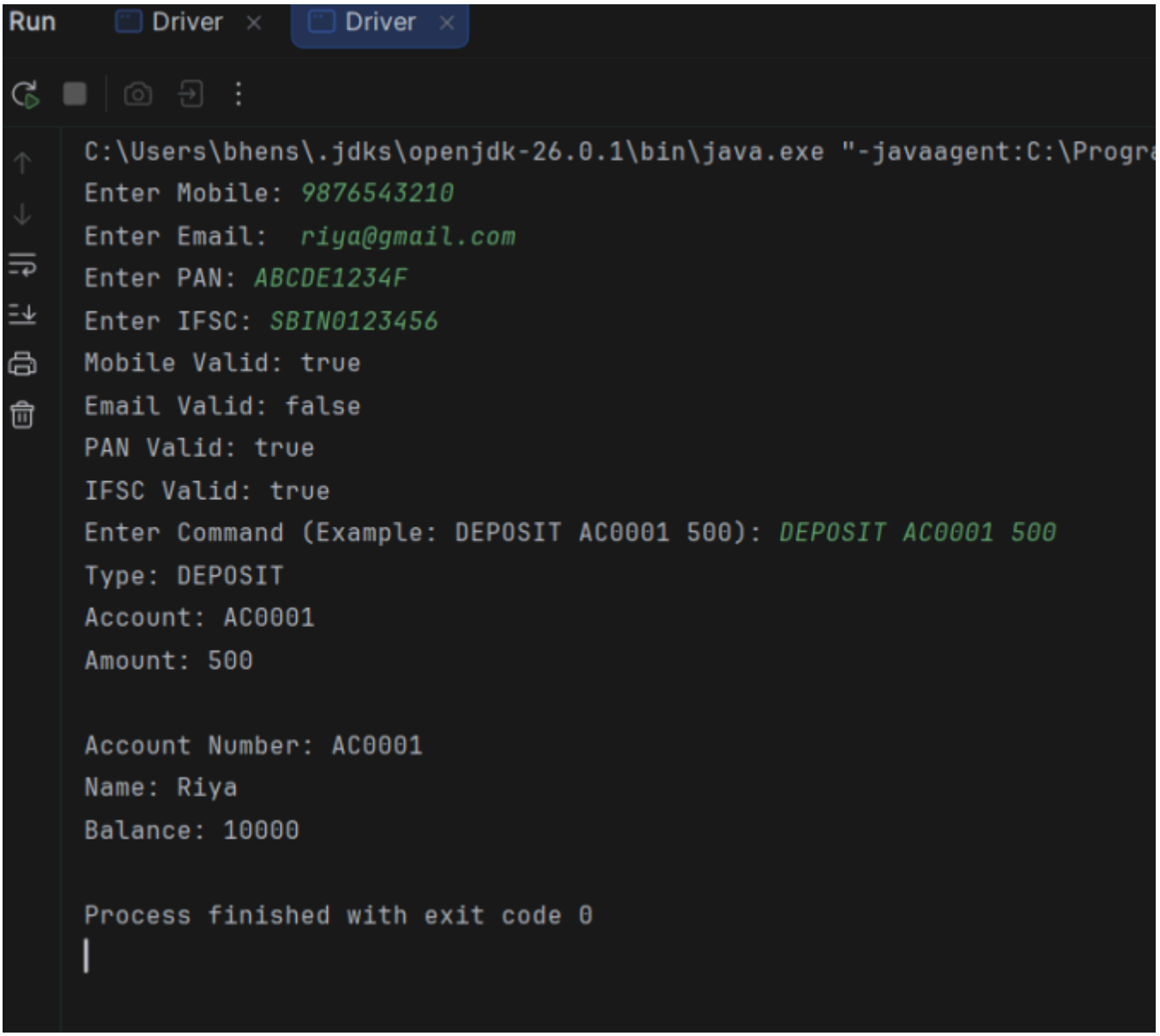
System.out.println(StatementFormatter.buildStatement(acc));

sc.close();

}

}
```

# OUTPUT



# KEY QUESTION:

**1. What is the difference between String, StringBuilder and StringBuffer, and when is each preferred?**

String is immutable, so its value cannot be changed after creation. StringBuilder is mutable and faster for modifying strings in a single-threaded program. StringBuffer is also mutable but thread-safe, so it is preferred in multi-threaded applications.

2**. What does the regular expression [6-9][0-9]{9} match, and why use anchors?**

The regular expression [6-9][0-9]{9} matches a 10-digit mobile number that starts with a digit from 6 to 9. Anchors ^ and $ ensure that the entire input matches the pattern and no extra characters are allowed.

**3. Why should input be validated at the boundary before it is used?**

Input should be validated before it is used to prevent invalid or malicious data from entering the program. This helps avoid errors, improves security, and ensures that the application works correctly with valid input only.

**LEARNING OUTCOME:**

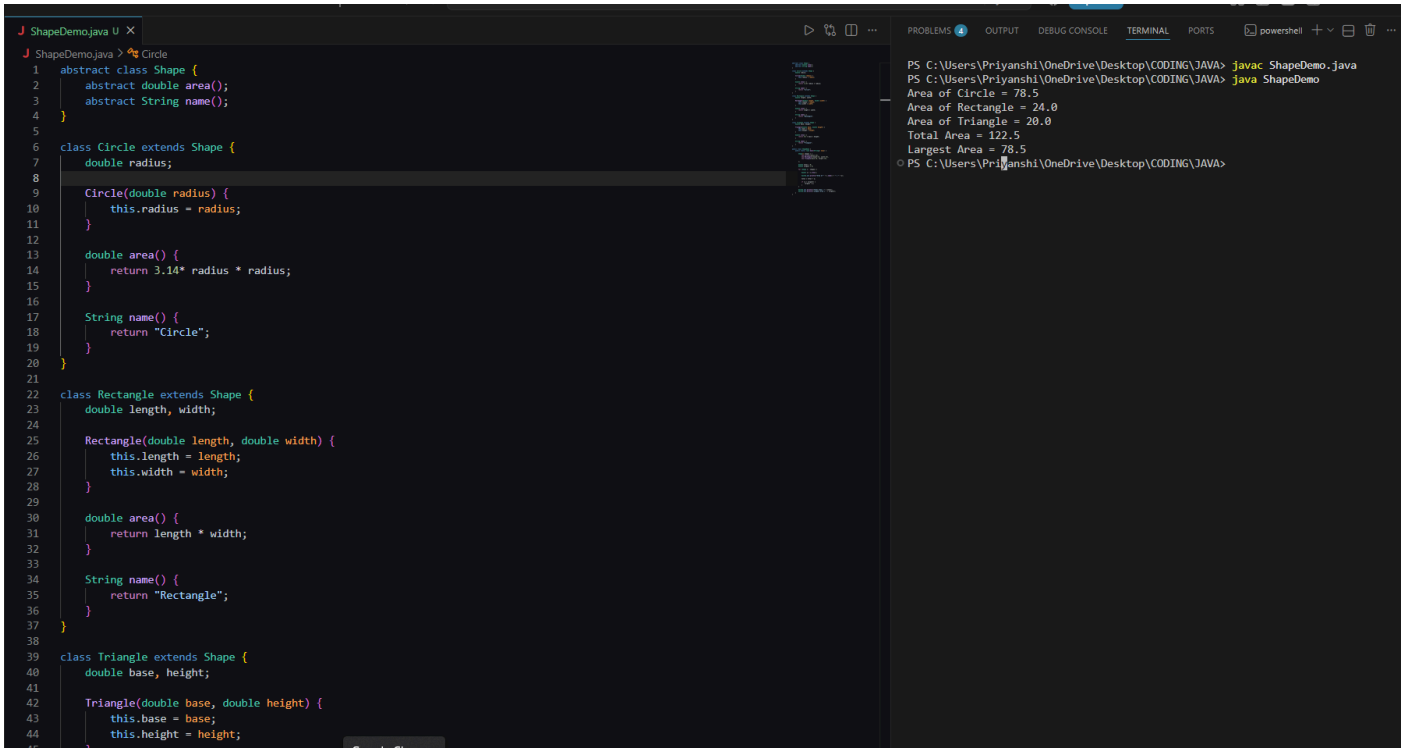
After completing this practical, I learned how to use Java strings, regular expressions, Pattern, Matcher, and StringBuilder for processing text and validating user input. I also learned to create and use classes, enums, records, and methods to build simple applications such as a password checker, chat log filter, template filler, and

MiniBank command parser. This practical improved my understanding of input validation, string manipulation, and object-oriented programming in Java.

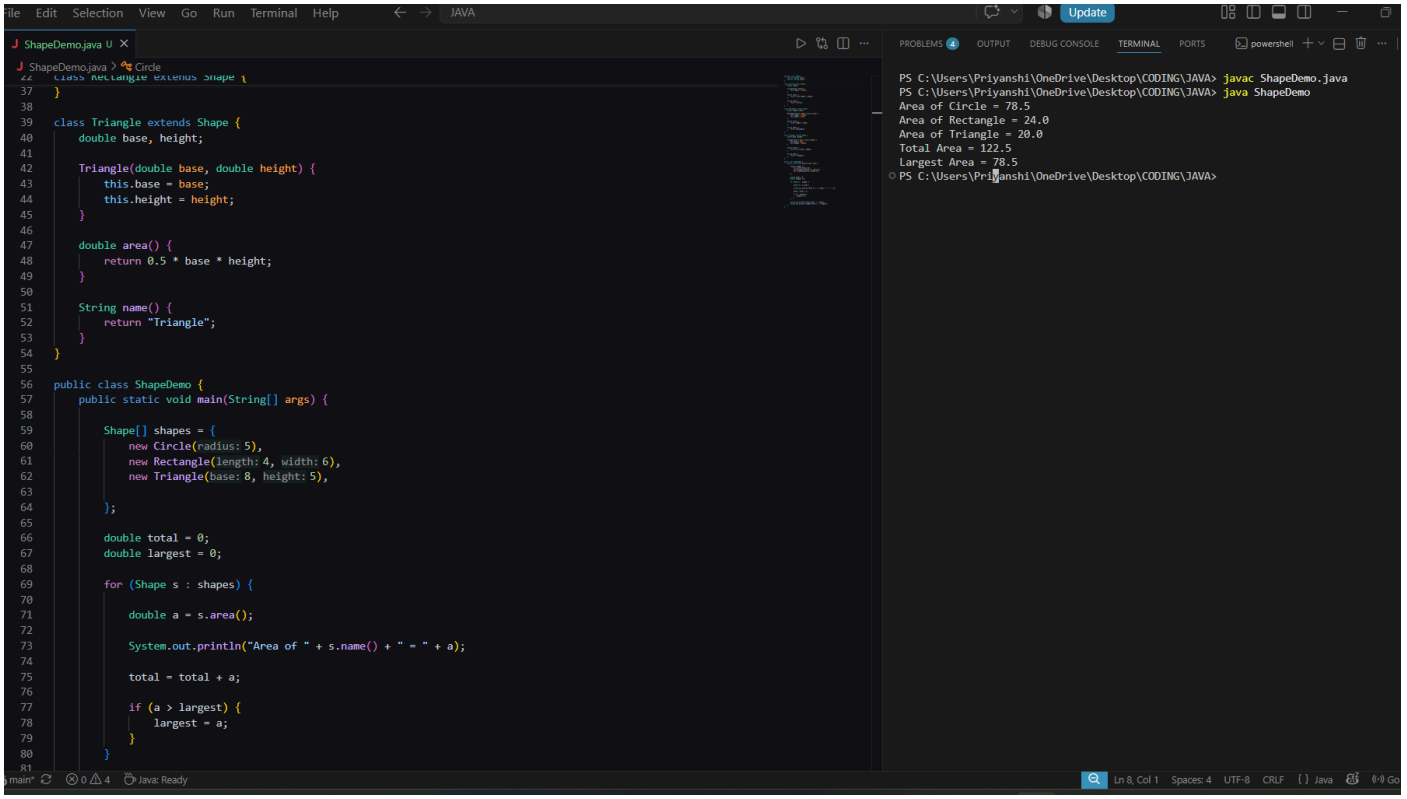
25IT006—Bhadiyadra Kavya

# PRACTICAL-05 | PART-A

## QUES-01

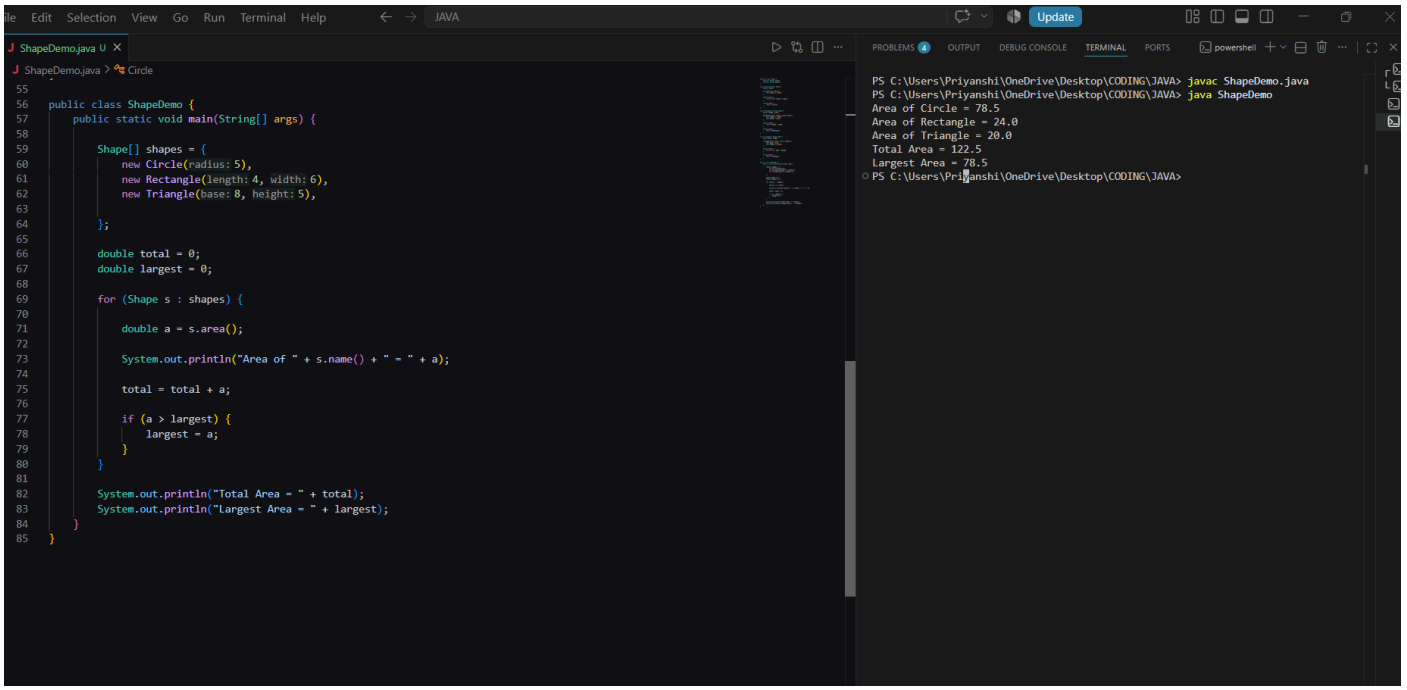


```
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> javac ShapeDemo.java
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> java ShapeDemo
Area of Circle = 78.5
Area of Rectangle = 24.0
Area of Triangle = 20.0
Total Area = 122.5
Largest Area = 78.5
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA>
```



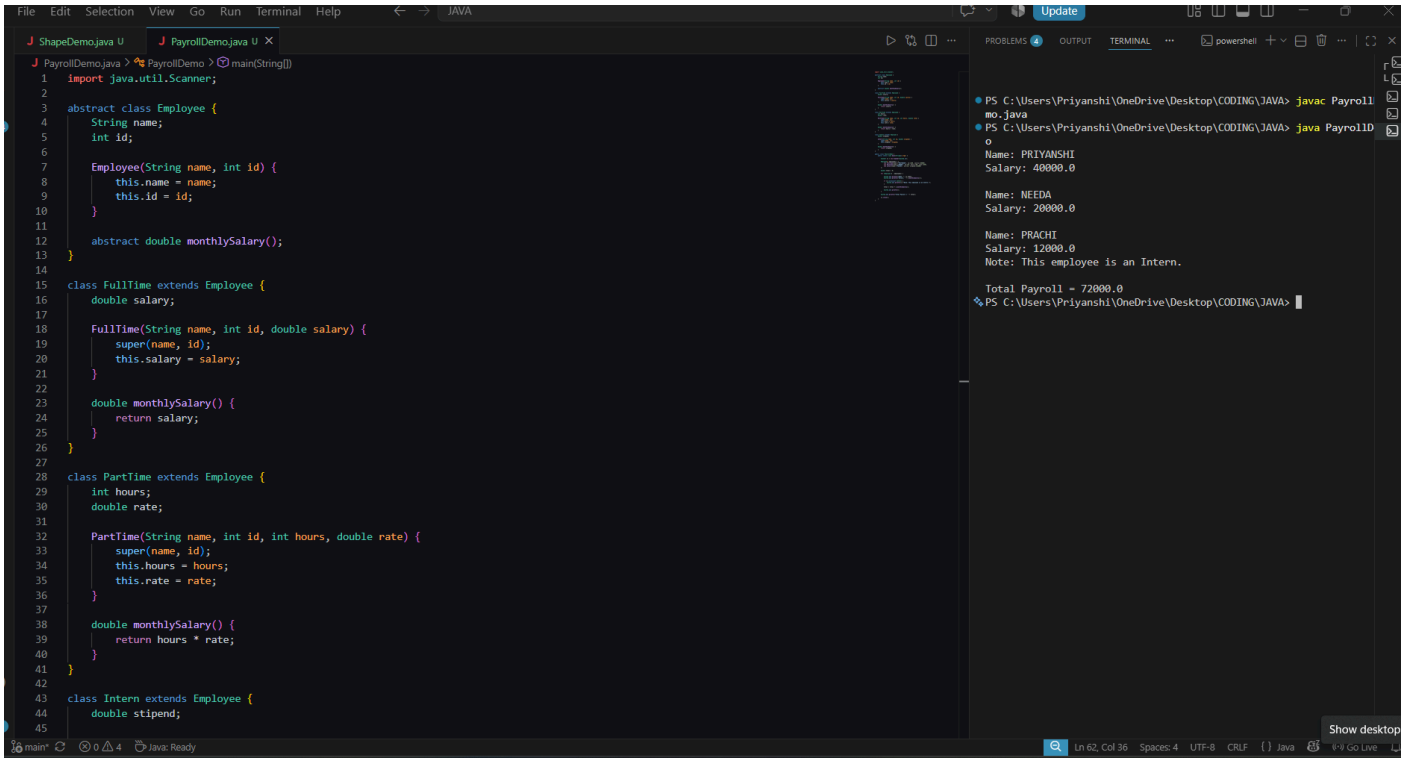
```
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> javac ShapeDemo.java
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> java ShapeDemo
Area of Circle = 78.5
Area of Rectangle = 24.0
Area of Triangle = 20.0
Total Area = 122.5
Largest Area = 78.5
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA>
```

# 25IT006—Bhadiyadra Kavya



**25IT006—Bhadiyadra Kavya**

# QUES-2



```java
import java.util.Scanner;

abstract class Employee {
    String name;
    int id;

    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    abstract double monthlySalary();
}

class FullTime extends Employee {
    double salary;

    FullTime(String name, int id, double salary) {
        super(name, id);
        this.salary = salary;
    }

    double monthlySalary() {
        return salary;
    }
}

class PartTime extends Employee {
    int hours;
    double rate;

    PartTime(String name, int id, int hours, double rate) {
        super(name, id);
        this.hours = hours;
        this.rate = rate;
    }

    double monthlySalary() {
        return hours * rate;
    }
}

class Intern extends Employee {
    double stipend;
```

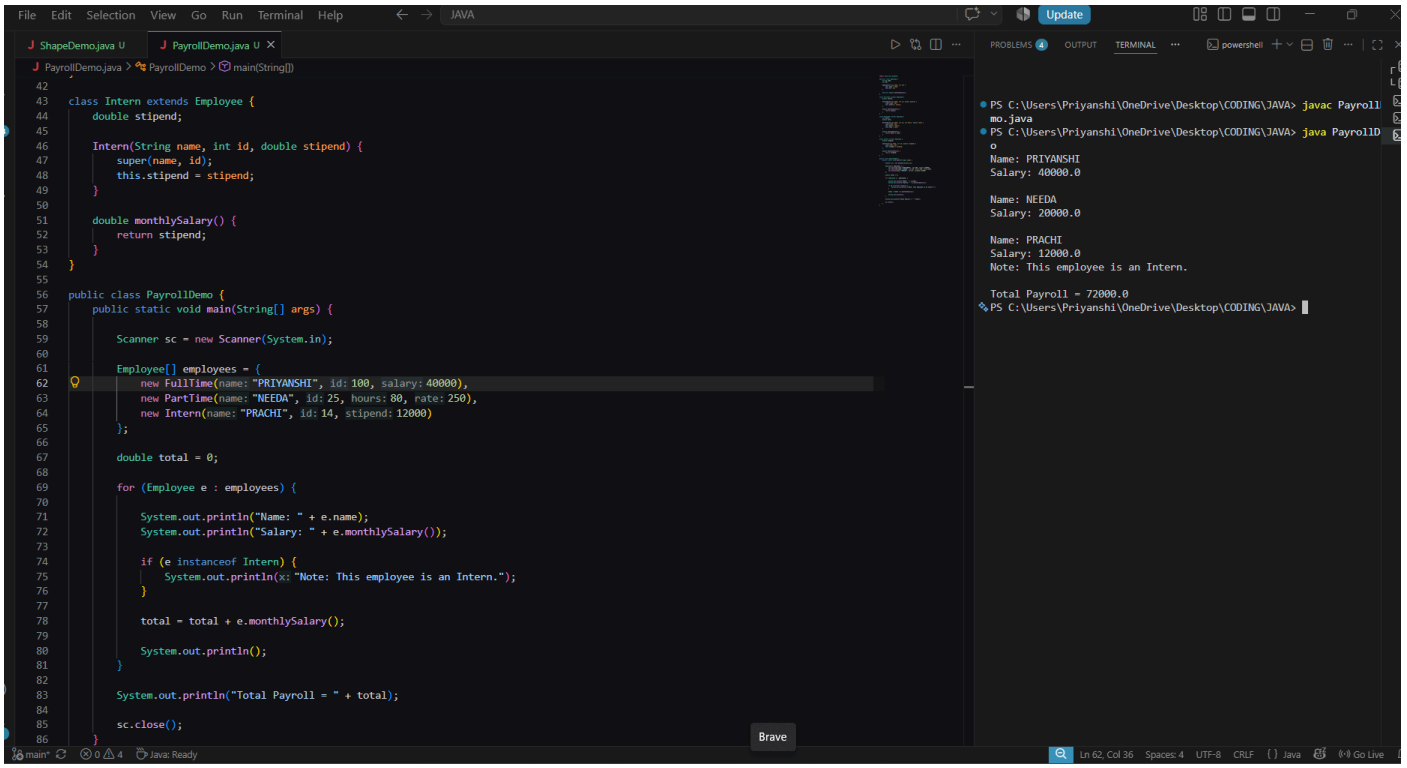
```
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> javac Payroll
mo.java
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> java Payroll
o
Name: PRIYANSHI
Salary: 40000.0

Name: NEEDA
Salary: 20000.0

Name: PRACHI
Salary: 12000.0
Note: This employee is an Intern.

Total Payroll = 72000.0
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA>
```

## 25IT006—Bhadiyadra Kavya



```
class Intern extends Employee {
    double stipend;

    Intern(String name, int id, double stipend) {
        super(name, id);
        this.stipend = stipend;
    }

    double monthlySalary() {
        return stipend;
    }
}

public class PayrollDemo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Employee[] employees = {
            new FullTime(name: "PRIYANSHI", id: 100, salary: 40000),
            new PartTime(name: "NEEDA", id: 25, hours: 80, rate: 250),
            new Intern(name: "PRACHI", id: 14, stipend: 12000)
        };

        double total = 0;

        for (Employee e : employees) {

            System.out.println("Name: " + e.name);
            System.out.println("Salary: " + e.monthlySalary());

            if (e instanceof Intern) {
                System.out.println(x: "Note: This employee is an Intern.");
            }

            total = total + e.monthlySalary();

            System.out.println();
        }

        System.out.println("Total Payroll = " + total);

        sc.close();
    }
}
```

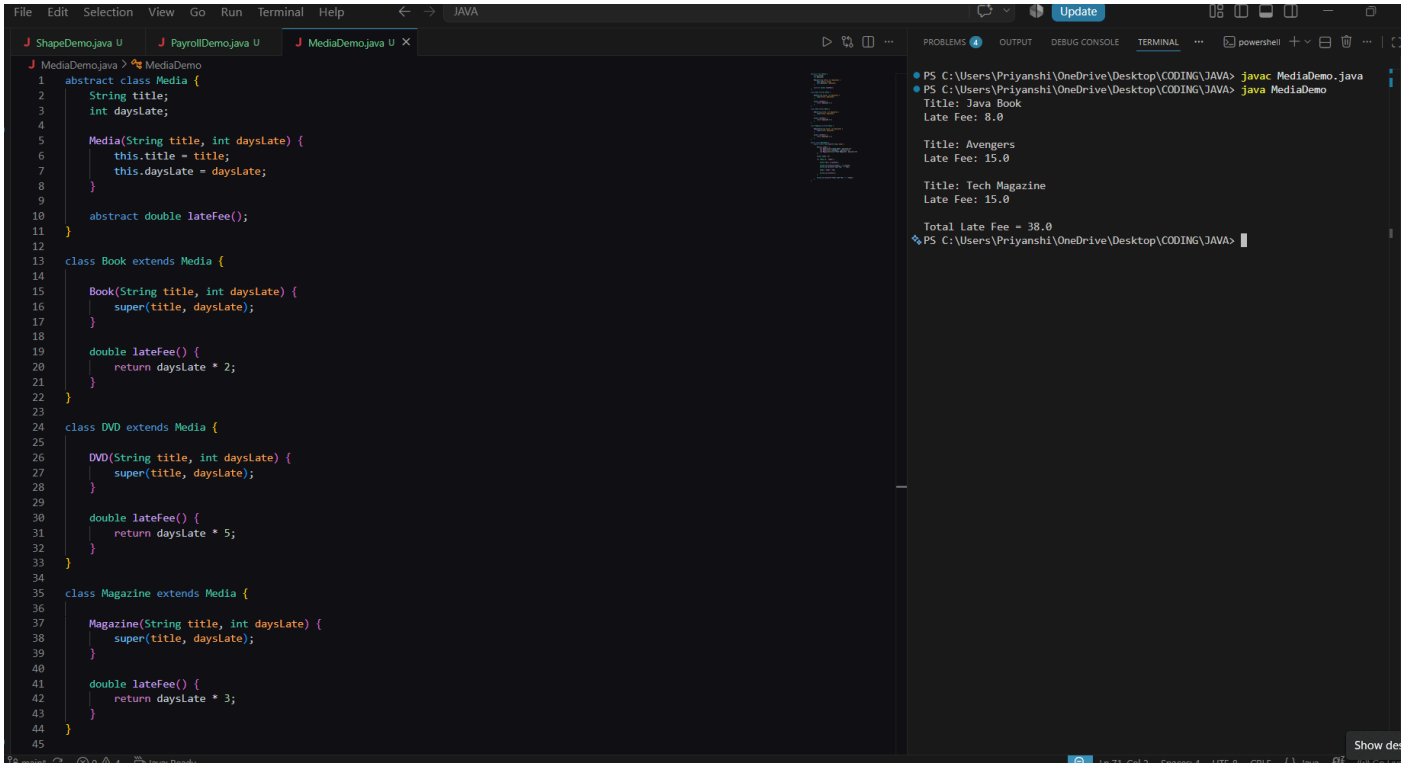
```
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> javac Payroll
mo.java
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> java PayrollD
o
Name: PRIYANSHI
Salary: 40000.0

Name: NEEDA
Salary: 20000.0

Name: PRACHI
Salary: 12000.0
Note: This employee is an Intern.

Total Payroll = 72000.0
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA>
```

# QUES-03



```
abstract class Media {
    String title;
    int daysLate;

    Media(String title, int daysLate) {
        this.title = title;
        this.daysLate = daysLate;
    }

    abstract double lateFee();
}

class Book extends Media {

    Book(String title, int daysLate) {
        super(title, daysLate);
    }

    double lateFee() {
        return daysLate * 2;
    }
}

class DVD extends Media {

    DVD(String title, int daysLate) {
        super(title, daysLate);
    }

    double lateFee() {
        return daysLate * 5;
    }
}

class Magazine extends Media {

    Magazine(String title, int daysLate) {
        super(title, daysLate);
    }

    double lateFee() {
        return daysLate * 3;
    }
}
```

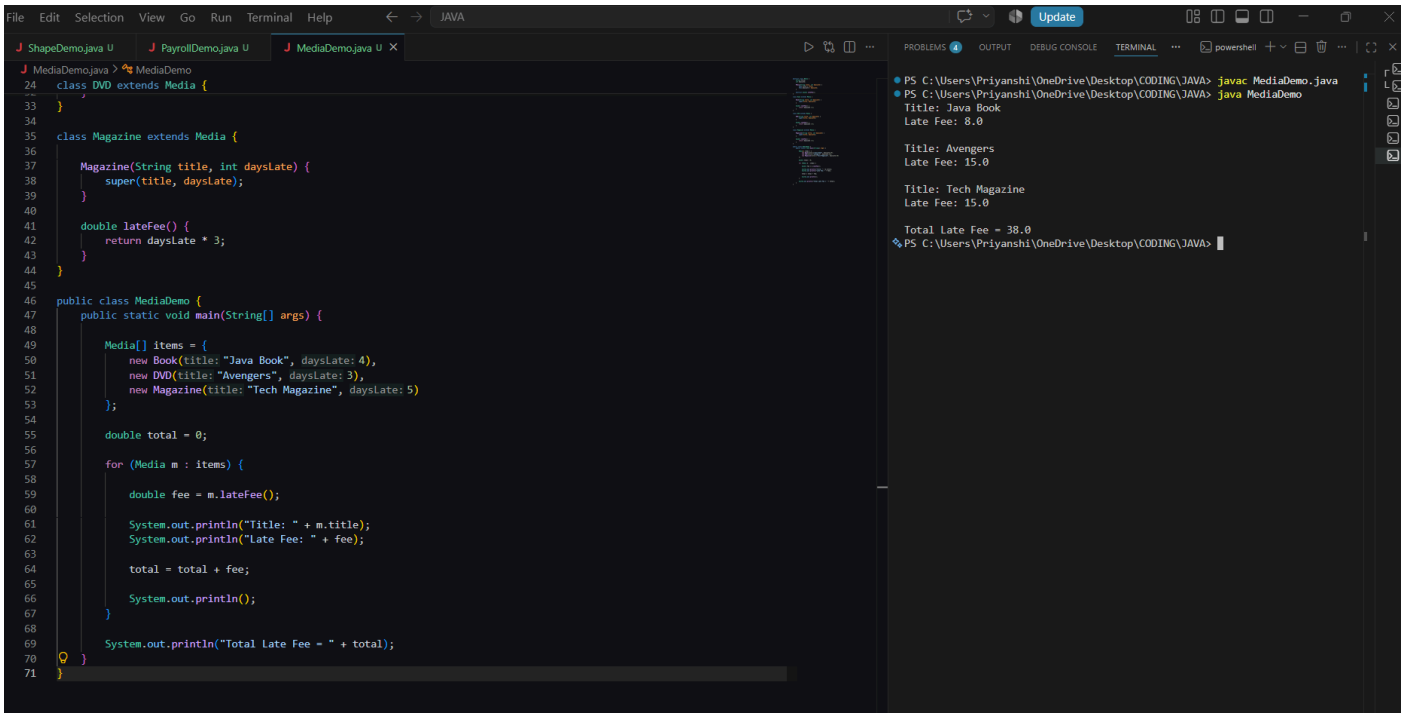
```
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> javac MediaDemo.java
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA> java MediaDemo
Title: Java Book
Late Fee: 8.0

Title: Avengers
Late Fee: 15.0

Title: Tech Magazine
Late Fee: 15.0

Total Late Fee = 38.0
PS C:\Users\Priyanshi\OneDrive\Desktop\CODING\JAVA>
```

# 25IT006—Bhadiyadra Kavya



# Practical : 6

1. **Remote control:** an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.

```
public interface Switchable {

   void on();

   void off();

   default void toggle() {
       off();
       on();
   }
}
```

```
class Fan implements Switchable {
   private boolean isOn = false;

   @Override
   public void on() {
       isOn = true;
       System.out.println("Fan is ON");
   }

   @Override
   public void off() {
       isOn = false;
       System.out.println("Fan is OFF");
   }
}
```

```

class Light implements Switchable {

   @Override
   public void on() {
       System.out.println("Light is ON");
   }

   @Override
```

```
    public void off() {
        System.out.println("Light is OFF");
    }
}
```

```
@FunctionalInterface
interface SwitchPolicy {

    boolean maySwitchOn(Switchable device, int hour);
}
```

```
public class RemoteControl {

    public static void main(String[] args) {
        Switchable[] devices = {
                new Fan(),
                new Light()
        };

        System.out.println("=== Toggling Devices ===");

        for (Switchable device : devices) {
            device.toggle();
        }

        SwitchPolicy anonymousPolicy = new SwitchPolicy() {

            @Override
            public boolean maySwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 22;
            }
        };


        // Lambda
        SwitchPolicy lambdaPolicy =
                (device, hour) -> hour >= 6 && hour <= 22;

        int hour = 20;

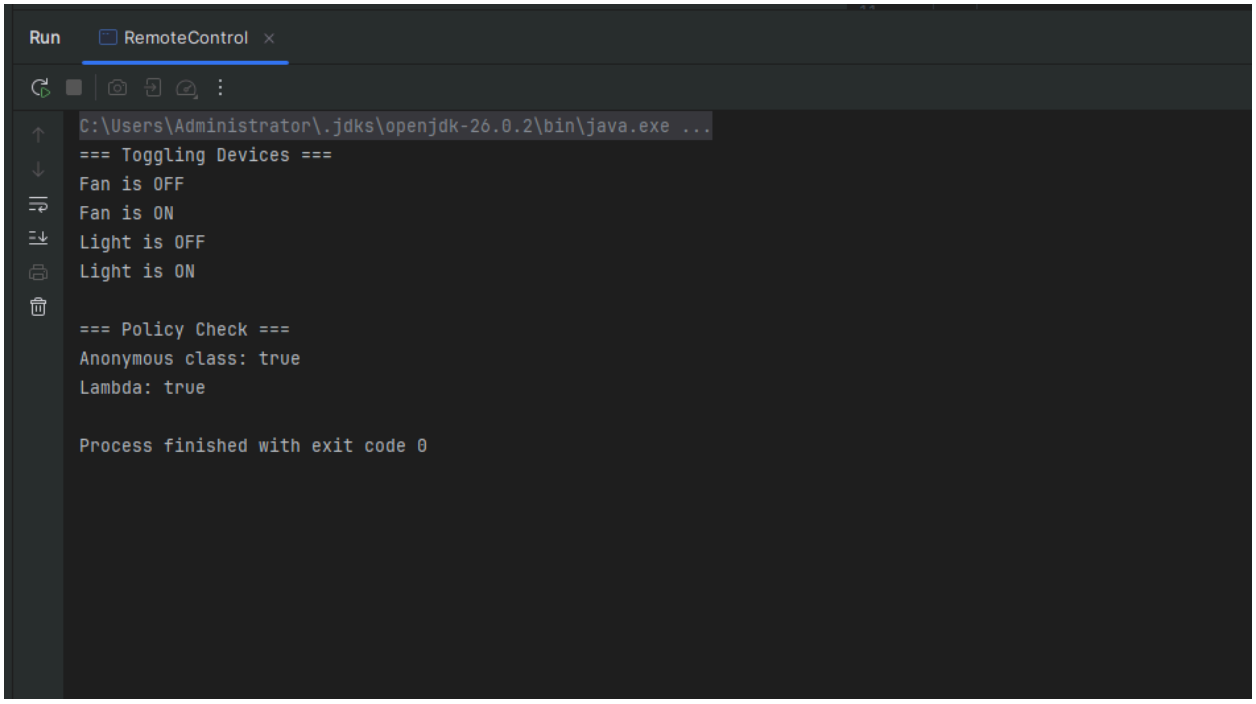
        System.out.println("\n=== Policy Check ===");

        System.out.println(
                "Anonymous class: " +
                        anonymousPolicy.maySwitchOn(devices[0], hour)
        );

        System.out.println(
```

```
                "Lambda: " +
                        lambdaPolicy.maySwitchOn(devices[1], hour)
        );
    }
}
```

# Output :




**2. Notification senders**: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.

```
@FunctionalInterface
interface Notifier {
    void send(String message);
}


// Marker interface
interface Urgent {
}


// Functional interface for urgent notifiers
```

```
@FunctionalInterface
interface UrgentNotifier extends Notifier, Urgent {
}
```

```
public class NotificationDemo {

   public static void main(String[] args) {

       // Email sender - marked as urgent
       UrgentNotifier email = message -> {
           System.out.println("Email: " + message);
       };


       // SMS sender - normal
       Notifier sms = message -> {
           System.out.println("SMS: " + message);
       };


       // Store both in an array
       Notifier[] senders = {
               email,
               sms
       };

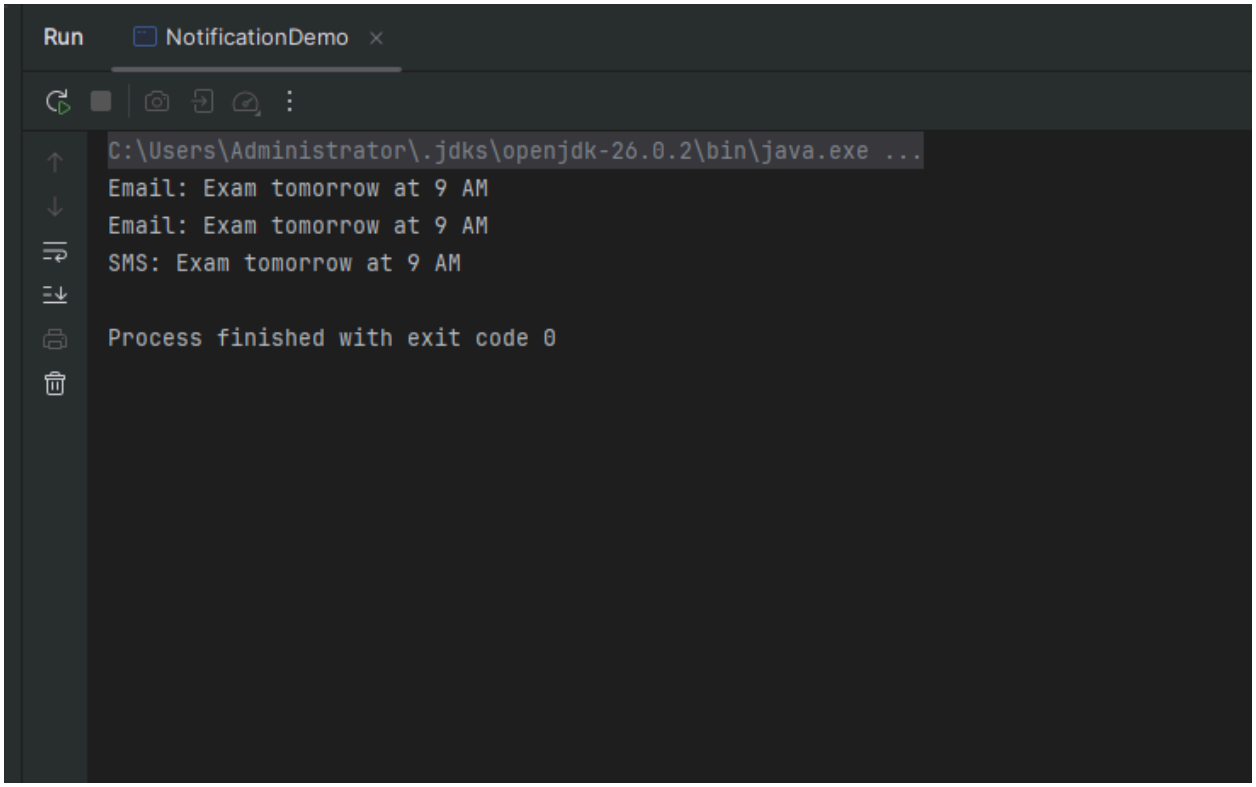

       // Message to broadcast
       String message = "Exam tomorrow at 9 AM";


       // Broadcast
       for (Notifier sender : senders) {

           // Send once
           sender.send(message);

           // Send again if urgent
           if (sender instanceof Urgent) {
               sender.send(message);
           }
       }
   }
}
```

# Output :




**3. (For Advanced Learners – home)** Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR.

```
package discount;

import java.util.List;

public class DiscountEngine {

   public static void applyDiscount(List<Double> prices, DiscountRule rule) {
       System.out.println("\nPrices after discount:");

       for (double price : prices) {
           double finalPrice = rule.apply(price);
           System.out.printf("%.2f -> %.2f%n", price, finalPrice);
       }
   }
}
```

```
package discount;

@FunctionalInterface
public interface DiscountRule {
   double apply(double price);
}
```

```
package discount;

import java.util.*;

public class Main {

   public static void main(String[] args) {

       Scanner sc = new Scanner(System.in);

       List<Double> prices = Arrays.asList(
               500.0,
               1000.0,
               1500.0,
               2000.0
       );

       System.out.println("=== Discount Engine ===");
       System.out.println("1. 10% Discount");
       System.out.println("2. 20% Discount");
       System.out.println("3. Flat ₹100 Discount");
       System.out.println("4. No Discount");

       System.out.print("Choose a discount rule: ");
       int choice = sc.nextInt();

       DiscountRule rule;

       switch (choice) {
           case 1:
               rule = price -> price * 0.90;
               break;

           case 2:
               rule = price -> price * 0.80;
               break;

           case 3:
               rule = price -> Math.max(0, price - 100);
               break;
```

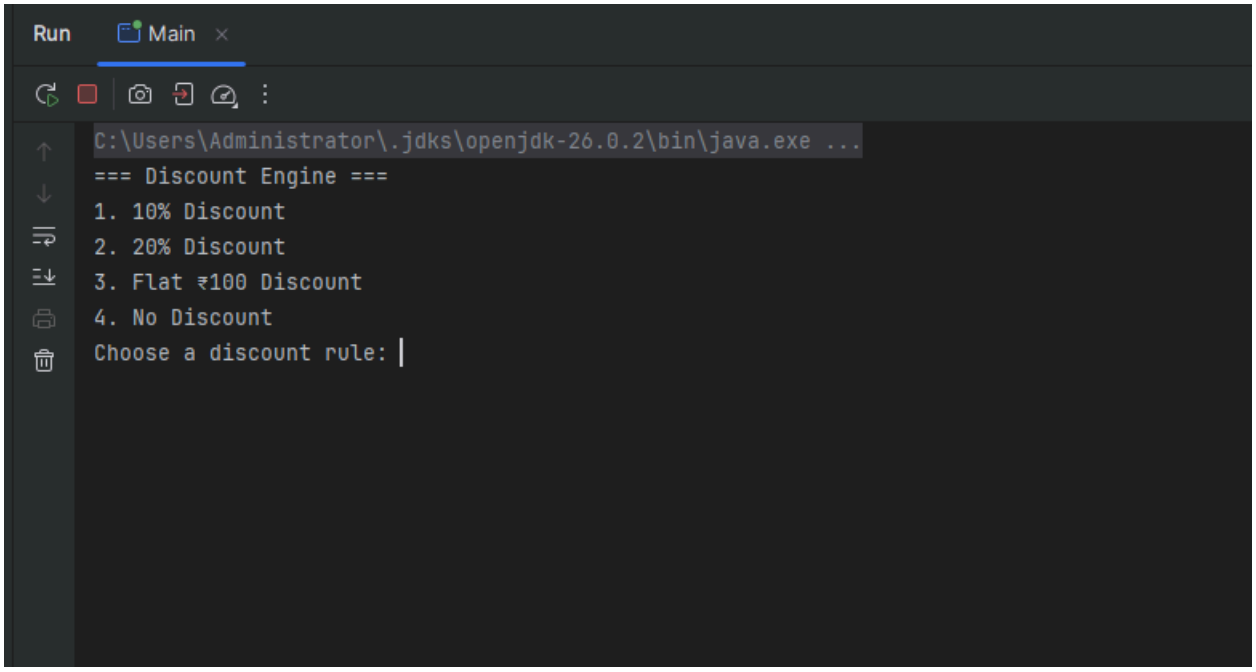
```
            case 4:
                rule = price -> price;
                break;

            default:
                System.out.println("Invalid choice.");
                sc.close();
                return;
        }

        DiscountEngine.applyDiscount(prices, rule);

        sc.close();
    }
}
```

## Output :



**25IT006**
**CEUE203**

# PRACTICAL 7

### Custom Annotations and Reflection in Java

## Aim

To create custom Java annotations and use reflection to inspect annotations at run time, validate object fields, and understand how frameworks discover annotated fields and methods.

## Part A — Practice Programs (Hour 1)

Practice the following programs in the lab-NN/ folder before beginning the MiniBank project. The third program is an advanced home exercise.

### 1. Form Validator

Define @NotBlank and @MaxLength(int) annotations for fields. Use reflection to inspect the annotations of a SignupForm object and return a list of validation errors.

**File: FormValidatorDemo.java**

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank { }

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class SignupForm {
    @NotBlank
    @MaxLength(20)
    private String username;

    @NotBlank
    @MaxLength(40)
    private String email;

    SignupForm(String username, String email) {
        this.username = username;
        this.email = email;
    }
}

class FormValidator {
    public static List<String> validate(Object obj) {
        List<String> errors = new ArrayList<>();

        for (Field field : obj.getClass().getDeclaredFields()) {
            field.setAccessible(true);
```

```
            try {
                Object value = field.get(obj);

                if (field.isAnnotationPresent(NotBlank.class)) {
                    if (value == null || value.toString().trim().isEmpty()) {
                        errors.add(field.getName() + " must not be blank");
                    }
                }

                MaxLength max = field.getAnnotation(MaxLength.class);
                if (max != null && value != null &&
                        value.toString().length() > max.value()) {
                    errors.add(field.getName() + " must be at most "
                            + max.value() + " characters");
                }
            } catch (IllegalAccessException e) {
                errors.add("Could not access field: " + field.getName());
            }
        }
        return errors;
    }
}

public class FormValidatorDemo {
    public static void main(String[] args) {
        SignupForm form = new SignupForm("   ",
                "a-very-long-email-address-that-exceeds-the-limit@example.com");

        List<String> errors = FormValidator.validate(form);
        if (errors.isEmpty()) {
            System.out.println("Form is valid.");
        } else {
            errors.forEach(System.out::println);
        }
    }
}
```

Sample output:

```
username must not be blank
email must be at most 40 characters
```

## 2. Mini Test Runner

Create a @Run annotation, mark no-argument methods with it, and use reflection to invoke only those methods. Print the number of methods executed.

### File: MiniTestRunnerDemo.java

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run { }

class SampleTests {
    @Run
    public void testAddition() {
        System.out.println("testAddition ran");
    }

    @Run
    public void testGreeting() {
        System.out.println("testGreeting ran");
```

```
    }

    public void helperMethod() {
        System.out.println("This method should not run");
    }
}

public class MiniTestRunnerDemo {
    public static void main(String[] args) throws Exception {
        SampleTests tests = new SampleTests();
        int count = 0;

        for (Method method : tests.getClass().getDeclaredMethods()) {
            if (method.isAnnotationPresent(Run.class)
                    && method.getParameterCount() == 0) {
                method.setAccessible(true);
                method.invoke(tests);
                count++;
            }
        }

        System.out.println("Methods run: " + count);
    }
}
```

Sample output:

```
testAddition ran
testGreeting ran
Methods run: 2
```

## 3. Advanced: Map CSV-Like Rows Using @Column

Define @Column(name) and populate an object's fields by matching a header row to a data row. This example supports String fields. If a column is missing, the field is left unchanged and a message is printed.

### File: ColumnMapperDemo.java

```
import java.lang.annotation.*;
import java.lang.reflect.Field;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class PersonRow {
    @Column(name = "full_name")
    private String name;

    @Column(name = "city")
    private String city;

    @Override
    public String toString() {
        return "PersonRow{name='" + name + "', city='" + city + "'}";
    }
}

class ColumnMapper {
    public static <T> T populate(T object, String[] headers, String[] row)
            throws IllegalAccessException {
        if (headers.length != row.length) {
            throw new IllegalArgumentException("Header and row sizes differ");
```

```
        }

        for (Field field : object.getClass().getDeclaredFields()) {
            Column column = field.getAnnotation(Column.class);
            if (column == null) continue;

            int index = -1;
            for (int i = 0; i < headers.length; i++) {
                if (headers[i].equals(column.name())) {
                    index = i;
                    break;
                }
            }

            if (index == -1) {
                System.out.println("Missing column: " + column.name());
                continue;
            }

            if (field.getType() == String.class) {
                field.setAccessible(true);
                field.set(object, row[index]);
            }
        }
        return object;
    }
}

public class ColumnMapperDemo {
    public static void main(String[] args) throws Exception {
        String[] headers = {"full_name"};
        String[] row = {"Asha Patel"};

        PersonRow person = ColumnMapper.populate(
                new PersonRow(), headers, row);
        System.out.println(person);
    }
}
```

Sample output:

```
Missing column: city
PersonRow{name='Asha Patel', city='null'}
```

**25IT006**
**CEUE203**

# PRACTICAL 7 — PROJECT WORK

## Part B — MiniBank (Hour 2)

Add metadata to MiniBank using custom annotations @Id, @Positive, and @MaxLength. The validator reads these annotations at run time with reflection and returns validation errors.

### Project structure

```
src/
└── model/
    ├── Account.java
    └── annotation/
        ├── Id.java
        ├── Positive.java
        ├── MaxLength.java
        └── AnnotationValidator.java
└── Main.java
```

### 1. Id.java — Marker Annotation

#### File: model/annotation/Id.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Id {
}
```

### 2. Positive.java

#### File: model/annotation/Positive.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Positive {
    String message() default "must be > 0";
}
```

### 3. MaxLength.java

#### File: model/annotation/MaxLength.java

```
package model.annotation;

import java.lang.annotation.ElementType;
```

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface MaxLength {
    int value();
}
```

## 4. Account.java — Model

### File: model/Account.java

```
package model;

import model.annotation.Id;
import model.annotation.Positive;
import model.annotation.MaxLength;

public class Account {
    @Id
    @MaxLength(12)
    private String accountNumber;

    @Positive(message = "balance must be greater than zero")
    private double balance;

    public Account(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    @Override
    public String toString() {
        return "Account{accountNumber='" + accountNumber
                + "', balance=" + balance + "}";
    }
}
```

## 5. AnnotationValidator.java — Reflection Validator

### File: model/annotation/AnnotationValidator.java

```
package model.annotation;

import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

public class AnnotationValidator {
    public static String[] validate(Object obj) {
        List<String> errors = new ArrayList<>();

        if (obj == null) {
            return new String[] {"Object must not be null"};
        }
```

```
        for (Field field : obj.getClass().getDeclaredFields()) {
            field.setAccessible(true);

            try {
                Object value = field.get(obj);

                Positive positive = field.getAnnotation(Positive.class);
                if (positive != null && value instanceof Number) {
                    if (((Number) value).doubleValue() <= 0) {
                        errors.add(field.getName() + " "
                                + positive.message());
                    }
                }

                MaxLength maxLength = field.getAnnotation(MaxLength.class);
                if (maxLength != null && value instanceof String) {
                    if (((String) value).length() > maxLength.value()) {
                        errors.add(field.getName() + " must be at most "
                                + maxLength.value() + " characters");
                    }
                }
            } catch (IllegalAccessException e) {
                errors.add("Unable to validate field: " + field.getName());
            }
        }

        return errors.toArray(new String[0]);
    }
}
```

## 6. Main.java — Test the Validator

### File: Main.java

```
import model.Account;
import model.annotation.AnnotationValidator;

public class Main {
    public static void main(String[] args) {
        Account account = new Account("ACC123", -500.0);

        System.out.println(account);
        String[] errors = AnnotationValidator.validate(account);

        if (errors.length == 0) {
            System.out.println("Account is valid.");
        } else {
            System.out.println("Validation errors:");
            for (String error : errors) {
                System.out.println("- " + error);
            }
        }
    }
}
```

Sample output:

```
Account{accountNumber='ACC123', balance=-500.0}
Validation errors:
- balance balance must be greater than zero
```

Note: The duplicated word in the sample message comes from combining the field name with the custom message. To make the output read more naturally, change the annotation message to "must be greater than zero"; the output will then be "balance must be greater than zero".

### 7. Required Existing-Code Annotations

Add @Override immediately above every method that overrides a superclass or interface method, such as toString(), equals(), hashCode(), and overridden subclass methods.

#### Example: WithdrawRule.java

```
@FunctionalInterface
public interface WithdrawRule {
    boolean canWithdraw(double balance, double amount);
}
```

@FunctionalInterface tells the compiler that the interface is intended to have exactly one abstract method. The compiler reports an error if this condition is not met.

### Expected output

```
Account{accountNumber='ACC123', balance=-500.0}
Validation errors:
- balance must be greater than zero
```

## Key Questions / Analysis / Interpretation

### 1. What are @Retention and @Target used for?

@Retention defines how long an annotation is available. SOURCE means it is discarded by the compiler, CLASS means it is stored in the class file but not necessarily available at run time, and RUNTIME makes it available through reflection. @Target restricts where an annotation can be used, such as FIELD, METHOD, TYPE, or PARAMETER.

### 2. What is reflection, and what does getDeclaredFields() return?

Reflection is a Java feature that lets a program inspect classes, fields, constructors, methods, and annotations while the program is running. getDeclaredFields() returns the fields declared directly in that class, including non-public fields, but not fields inherited from its parent class.

### 3. How do annotations plus reflection let a framework validate any object?

Annotations attach rules or metadata to fields. At run time, reflection discovers each field, checks which annotations it has, reads the field value, and applies the corresponding rule. Because the validator examines the object's class rather than relying on a specific model type, the same validator can validate different classes that use the supported annotations.

## Conclusion

Custom annotations and reflection were used to attach validation rules to fields and check those rules at run time. The MiniBank example demonstrates how frameworks can inspect metadata and validate objects automatically.

# PRACTICAL 8

## Part A

1. Guarded calculator: read two numbers and an operator; throw a custom

DivideByZeroException and catch invalid-number input separately, printing a clear

message each time; loop until a valid calculation succeeds and use finally to log each

attempt.

## CODE:

```
import java.util.Scanner;


// Custom exception
class DivideByZeroException extends Exception {
   public DivideByZeroException(String message) {
       super(message);
```

```
    }
}


public class GuardedCalculator {


    // Method to perform calculation
    static double calculate(double a, double b, char operator)
            throws DivideByZeroException {


        switch (operator) {
            case '+':
                return a + b;


            case '-':
                return a - b;


            case '*':
                return a * b;


            case '/':
                if (b == 0) {
                    throw new DivideByZeroException(
                            "Cannot divide by zero."
                    );
                }
```

```
            return a / b;


        default:
            throw new IllegalArgumentException(
                    "Invalid operator."
            );
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    boolean success = false;

    while (!success) {

        try {
            System.out.print("Enter first number: ");
            double num1 = Double.parseDouble(sc.nextLine());

            System.out.print("Enter second number: ");
            double num2 = Double.parseDouble(sc.nextLine());

            System.out.print("Enter operator (+, -, *, /): ");
            char operator = sc.nextLine().charAt(0);
```

```
            double result = calculate(num1, num2, operator);

            System.out.println("Result = " + result);

            // Calculation succeeded
            success = true;

        } catch (NumberFormatException e) {
            System.out.println(
                    "Invalid number input. Please enter valid numbers."
            );

        } catch (DivideByZeroException e) {
            System.out.println(
                    "Calculation error: " + e.getMessage()
            );

        } catch (IllegalArgumentException e) {
            System.out.println(
                    "Input error: " + e.getMessage()
            );

        } finally {
            // Runs after every attempt
```

```
                System.out.println("Attempt logged.");

                System.out.println("---------------------");

            }

        }



        sc.close();

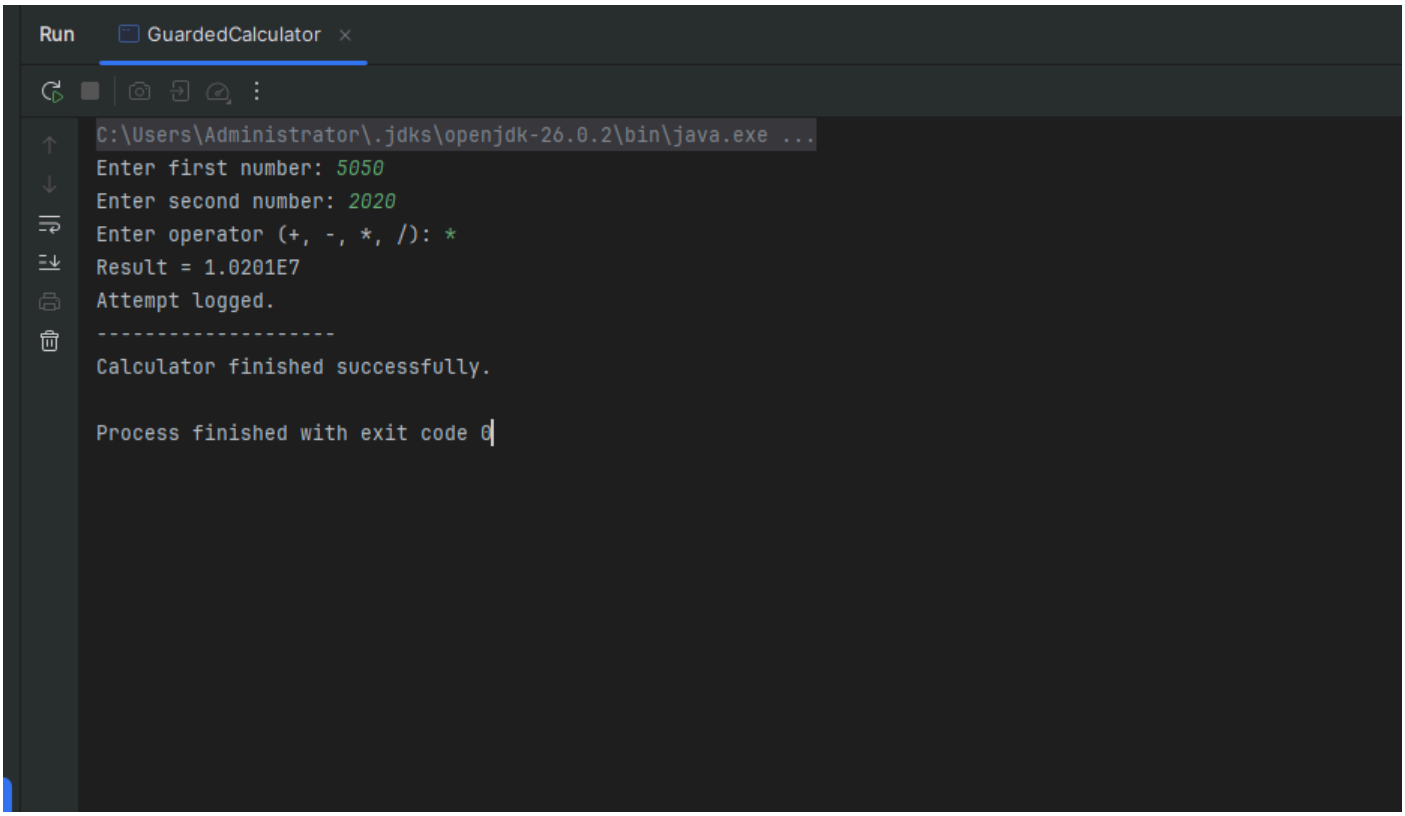
        System.out.println("Calculator finished successfully.");

    }

}
```

# OUTPUT:

# 25IT006 KAVYA BHADIYADRA 4 CEUE203–Object Oriented Programming




**2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0.**
**Process a list of requests, catching and reporting each failure without stopping the run.**

```
import java.util.*;

class OutOfStockException extends Exception {
   private int shortfall;
```

# 25IT006 KAVYA BHADIYADRA 4 CEUE203–Object Oriented Programming

```
    public OutOfStockException(int shortfall) {
        super("Out of stock. Shortfall = " + shortfall);
        this.shortfall = shortfall;
    }

    public int getShortfall() {
        return shortfall;
    }
}

class InvalidQuantityException extends Exception {

    public InvalidQuantityException(String message) {
        super(message);
    }
}

class Warehouse {

    private Map<String, Integer> stock = new HashMap<>();

    public void addStock(String item, int quantity) {
        stock.put(item, stock.getOrDefault(item, 0) + quantity);
    }

    public void issue(String item, int qty)
            throws OutOfStockException, InvalidQuantityException {

        if (qty <= 0) {
            throw new InvalidQuantityException(
                    "Quantity must be greater than 0."
            );
        }

        int available = stock.getOrDefault(item, 0);

        if (qty > available) {
            int shortfall = qty - available;
            throw new OutOfStockException(shortfall);
        }

        stock.put(item, available - qty);

        System.out.println("Issued " + qty + " " + item);
    }
}

public class Main {

    public static void main(String[] args) {

        Warehouse warehouse = new Warehouse();
```

```
        warehouse.addStock("Laptop", 10);
        warehouse.addStock("Mouse", 20);
        warehouse.addStock("Keyboard", 5);

        String[][] requests = {
                {"Laptop", "3"},
                {"Mouse", "0"},
                {"Keyboard", "10"},
                {"Mouse", "5"},
                {"Laptop", "-2"},
                {"Keyboard", "2"}
        };

        for (String[] request : requests) {

            String item = request[0];
            int qty = Integer.parseInt(request[1]);

            try {
                warehouse.issue(item, qty);

            } catch (OutOfStockException e) {

                System.out.println(
                        "Failed: " + item + " - " + e.getMessage()
                );

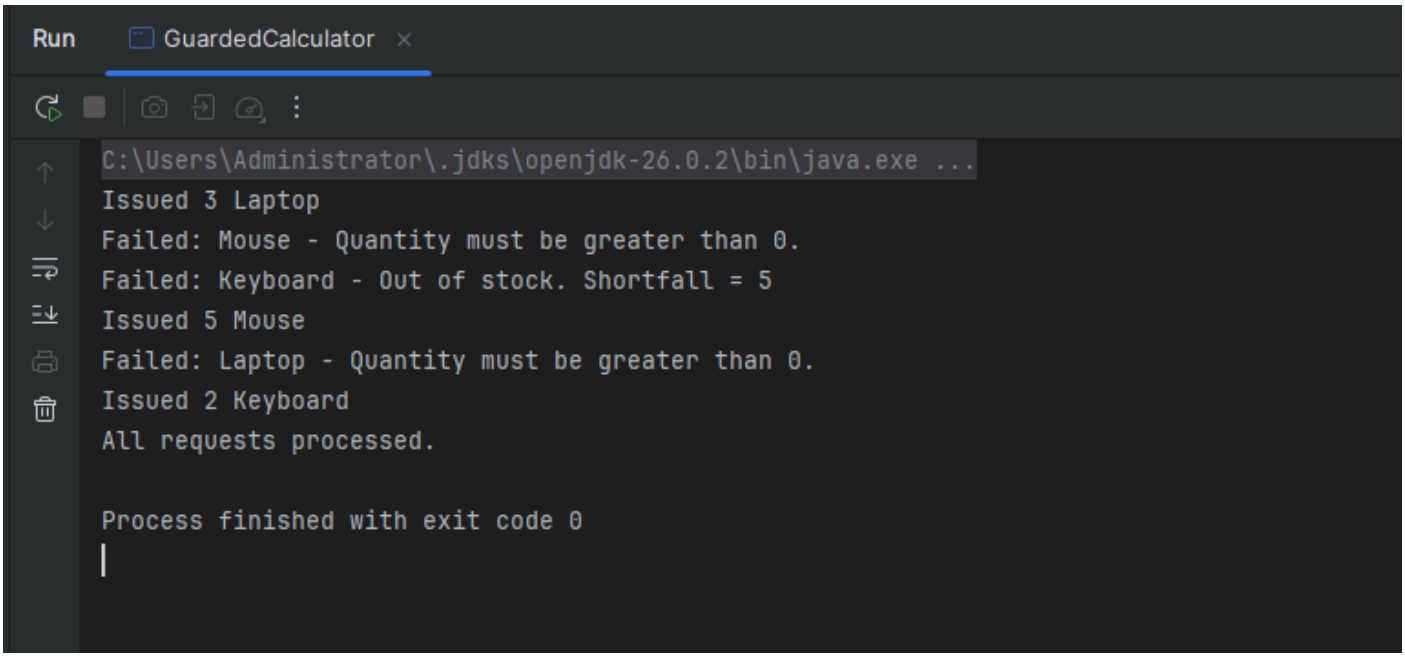
            } catch (InvalidQuantityException e) {

                System.out.println(
                        "Failed: " + item + " - " + e.getMessage()
                );
            }
        }

        System.out.println("All requests processed.");
    }
}
```

# Output:

# 25IT006 KAVYA BHADIYADRA 4 CEUE203–Object Oriented Programming




**3. (For Advanced Learners – home) Create an AutoCloseable resource and use try-with-**
**resources to prove it always closes — even when an exception is thrown inside the block**

**— and that the original error is the one reported. Decide what to model as the resource.**

```
class MyResource implements AutoCloseable {

    public MyResource() {
        System.out.println("Resource opened.");
    }

    public void use() {
        System.out.println("Using resource...");
        throw new RuntimeException("Original error inside resource.");
    }

    @Override
    public void close() {
```

```
        System.out.println("Resource closed.");
    }
}

public class GuardedCalculator {

    public static void main(String[] args) {

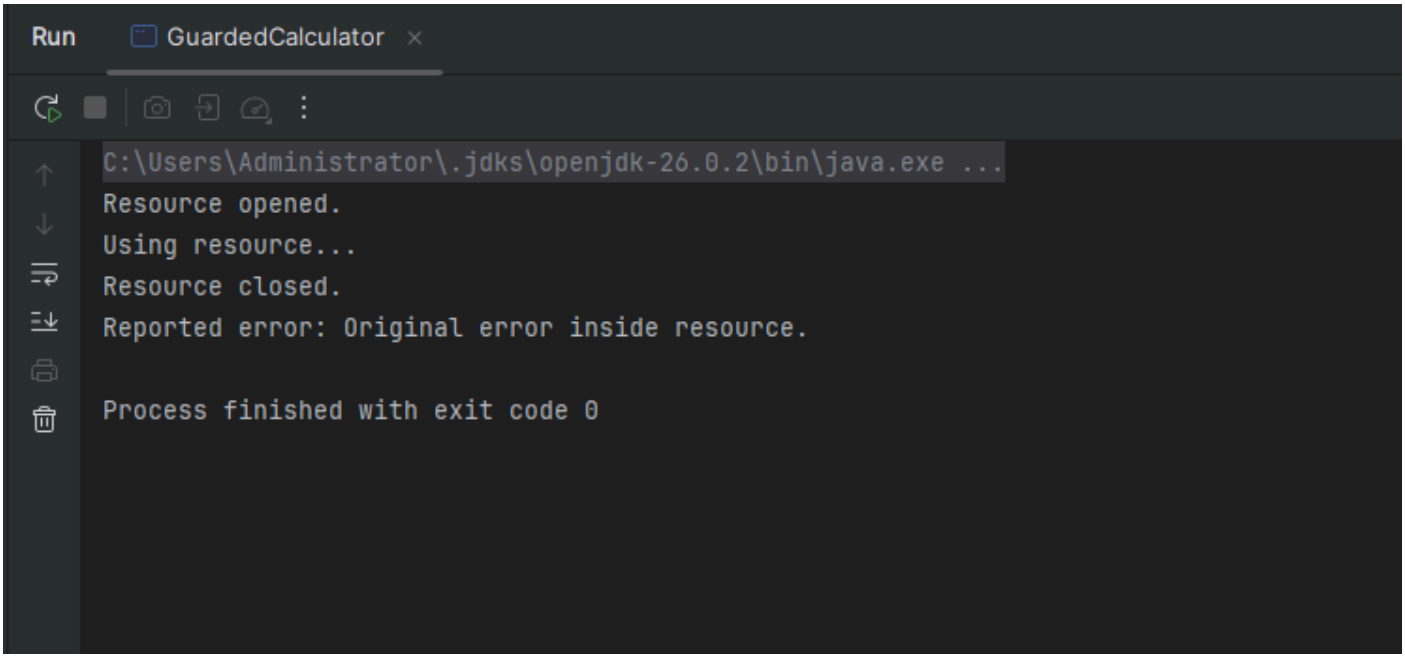
        try (MyResource resource = new MyResource()) {

            resource.use();

        } catch (Exception e) {

            System.out.println("Reported error: " + e.getMessage());
        }
    }
}
```

# Output:



# PRACTICAL 9

## Part A

1. **Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.**

## CODE:

```
import java.util.Scanner;
class Counter {
int count = 0;

   void increment() {
       count++;
   }

   synchronized void synchronizedIncrement() {
       count++;
   }
}

class CounterThread extends Thread {
   Counter c;
int times; boolean
   sync;

   CounterThread(Counter c, int times, boolean sync) {
       this.c = c;
this.times = times; this.sync
       = sync;
   }

public void run() {
       for (int i = 0; i < times; i++) {
           if (sync)
c.synchronizedIncrement();
```

```
                                    else c.increment();

                    }
                }
            }


public class CounterRace {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of threads: ");
        int n = sc.nextInt();

        System.out.print("Enter increments per thread: ");
        int times = sc.nextInt();

        Counter c1 = new Counter();
        CounterThread[] t1 = new CounterThread[n];

        for (int i = 0; i < n; i++) {
            t1[i] = new CounterThread(c1, times, false);
            t1[i].start();
        }

        for (int i = 0; i < n; i++)
            t1[i].join();

        System.out.println("Without Synchronization: " + c1.count);
        System.out.println("Expected Count: " + (n * times));

        Counter c2 = new Counter();
        CounterThread[] t2 = new CounterThread[n];

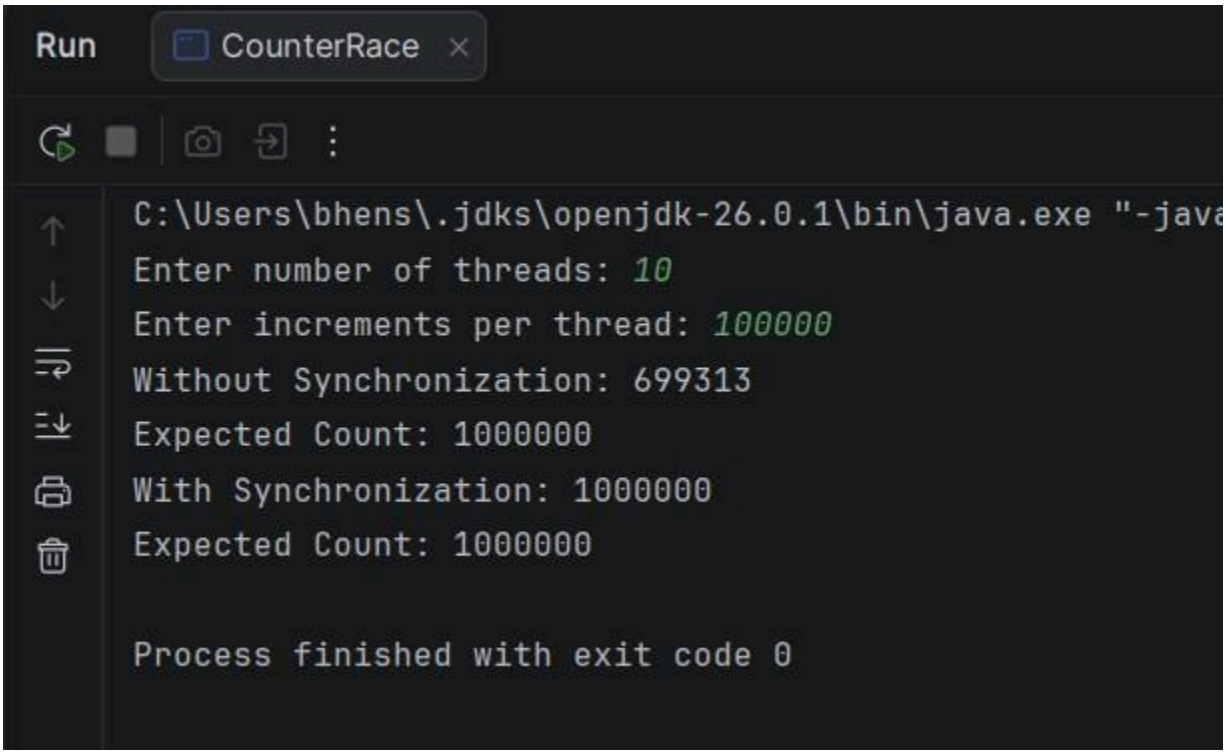
        for (int i = 0; i < n; i++) {
            t2[i] = new CounterThread(c2, times, true);
            t2[i].start();
        }

        for (int i = 0; i < n; i++)
            t2[i].join();

        System.out.println("With Synchronization: " + c2.count);
        System.out.println("Expected Count: " + (n * times));

        sc.close();
    }
}
```

# OUTPUT:




**2. Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.**

# CODE:

```
import java.util.Scanner;

class SeatBooking {
   int seatsLeft = 5;

boolean book() {
       if (seatsLeft > 0) {
           try {
               Thread.sleep(10);
} catch (InterruptedException e) {
           }
seatsLeft--; return
           true;
       }
return false;
```

```
    synchronized boolean synchronizedBook() {
        if (seatsLeft > 0) {
            seatsLeft--;
            return true;
        }
        return false;
    }
}

class BookingThread extends Thread {
    SeatBooking booking;
    boolean sync;

    BookingThread(SeatBooking booking, boolean sync) {
        this.booking = booking;
        this.sync = sync;
    }

    public void run() {
        if (sync) {
            if (booking.synchronizedBook())
                System.out.println(getName() + " booked a seat");
            else
                System.out.println(getName() + " could not book");
        } else {
            if (booking.book())
                System.out.println(getName() + " booked a seat");
            else
                System.out.println(getName() + " could not book");
        }
    }
}

public class SeatBookingRace {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of booking threads: ");
        int n = sc.nextInt();

        SeatBooking b1 = new SeatBooking();
        BookingThread[] t1 = new BookingThread[n];
        System.out.println("\nWithout Synchronization:");

        for (int i = 0; i < n; i++) {

            t1[i] = new BookingThread(b1, false);
            t1[i].setName("Thread-" + (i + 1));
            t1[i].start();
        }

        for (int i = 0; i < n; i++)
            t1[i].join();

        System.out.println("Seats Left: " + b1.seatsLeft);

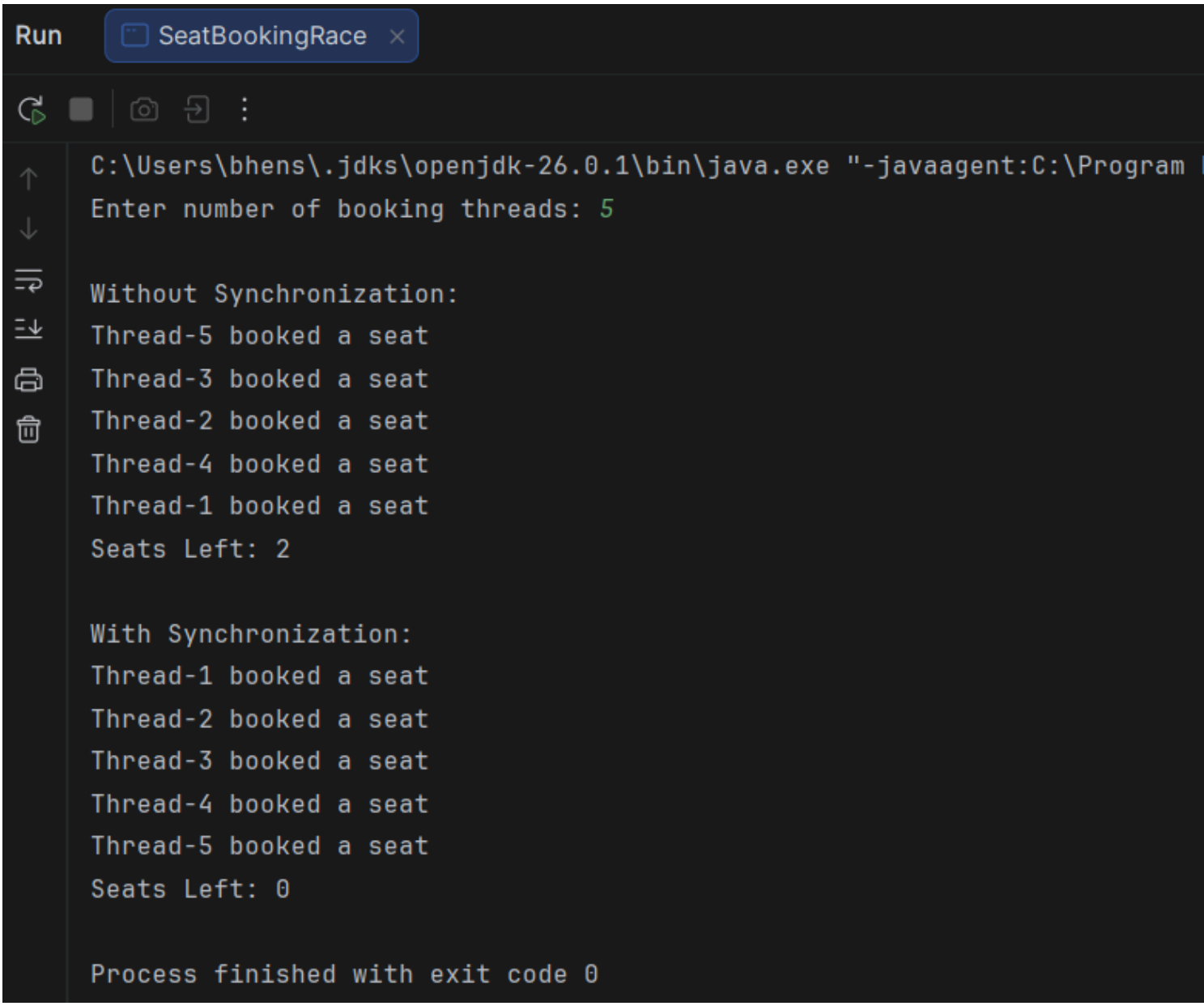
        SeatBooking b2 = new SeatBooking();
        BookingThread[] t2 = new BookingThread[n];

        System.out.println("\nWith Synchronization:");
```

```
for (int i = 0; i < n; i++) {
t2[i] = new BookingThread(b2, true);
            t2[i].setName("Thread-" + (i + 1));
            t2[i].start();
        }

        for (int i = 0; i < n; i++)
            t2[i].join();
System.out.println("Seats Left: " + b2.seatsLeft);
        sc.close();
    }
}
```

# OUTPUT:



# 3. Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare decide how to measure the difference.

## CODE:

```
import java.util.Scanner;

class SharedTotal {

    int total = 0;

    void add(int value) {
        total += value;
    }

    synchronized void synchronizedAdd(int value) {
        total += value;
    }
}

class SumThread extends Thread {
    int[] arr;
    int start;
    int end;
    SharedTotal total;
    boolean sync;

    SumThread(int[] arr, int start, int end, SharedTotal total, boolean sync) {
        this.arr = arr;
        this.start = start;
        this.end = end;
        this.total = total;
        this.sync = sync;
    }

    public void run() {
        for (int i = start; i < end; i++) {
            if (sync)
                total.synchronizedAdd(arr[i]);
            else
                total.add(arr[i]);
        }
    }
}

public class SharedArraySum {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter array size: ");
        int n = sc.nextInt();
```

```
        int[] arr = new int[n];

        System.out.println("Enter array elements:");

        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        System.out.print("Enter number of threads: ");
        int threads = sc.nextInt();

        int expected = 0;

        for (int i = 0; i < n; i++)
            expected += arr[i];

        SharedTotal t1Total = new SharedTotal();
        SumThread[] t1 = new SumThread[threads];

        int part = n / threads;

        for (int i = 0; i < threads; i++) {
            int start = i * part;
            int end = (i == threads - 1) ? n : start + part;

            t1[i] = new SumThread(arr, start, end, t1Total, false);
            t1[i].start();
        }

        for (int i = 0; i < threads; i++)
            t1[i].join();

        System.out.println("\nExpected Sum: " + expected); System.out.println("Without
        Synchronization: " + t1Total.total);

        SharedTotal t2Total = new SharedTotal();
        SumThread[] t2 = new SumThread[threads];

        for (int i = 0; i < threads; i++) {
            int start = i * part;
            int end = (i == threads - 1) ? n : start + part;

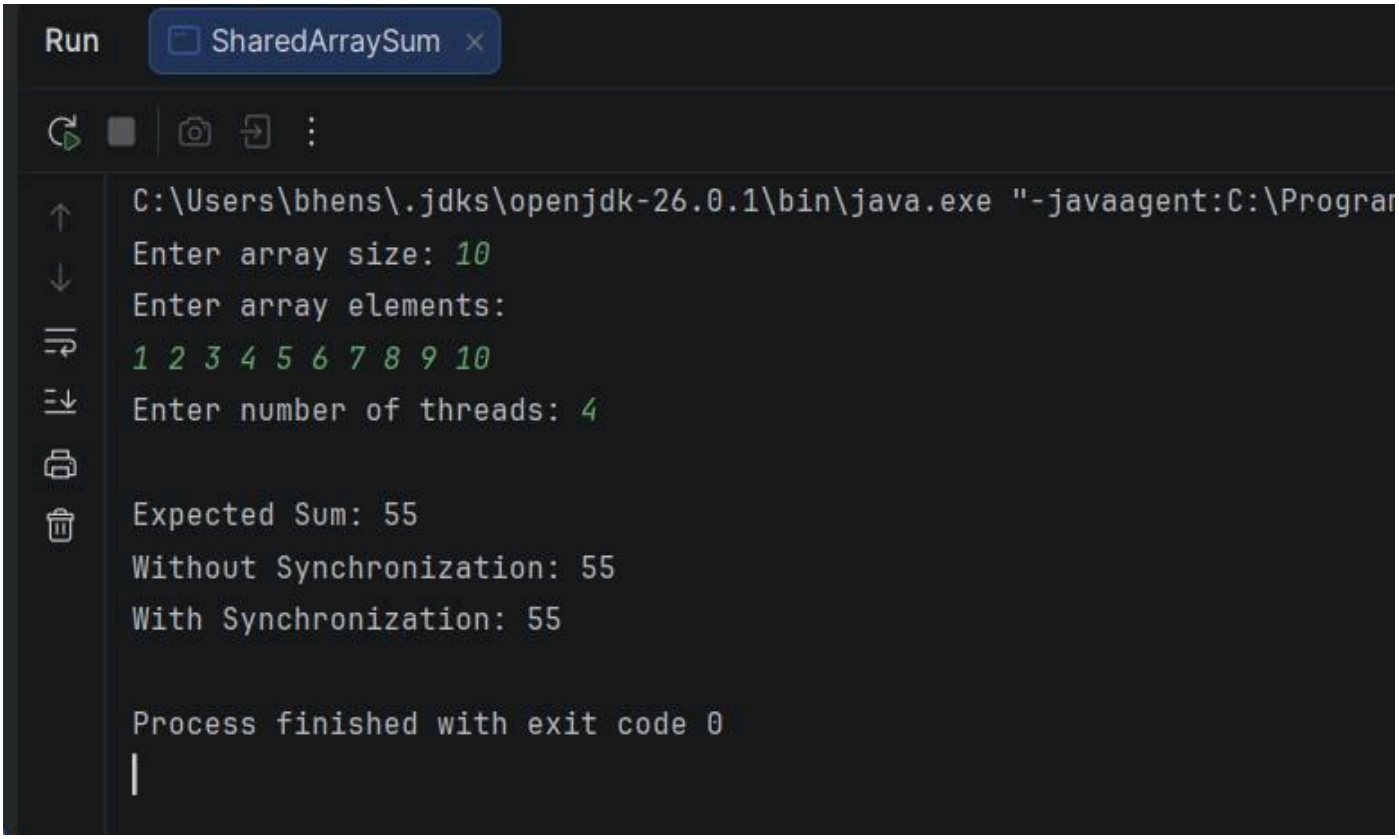
            t2[i] = new SumThread(arr, start, end, t2Total, true);
            t2[i].start();
        }

        for (int i = 0; i < threads; i++)
            t2[i].join();

        System.out.println("With Synchronization: " + t2Total.total);

        sc.close();
    }
}
```

# OUTPUT:


Run
SharedArraySum
C:\Users\bhens\.jdks\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Prograr
Enter array size: 10
Enter array elements:
1 2 3 4 5 6 7 8 9 10
Enter number of threads: 4
Expected Sum: 55
Without Synchronization: 55
With Synchronization: 55
Process finished with exit code 0

# PRACTICAL 10

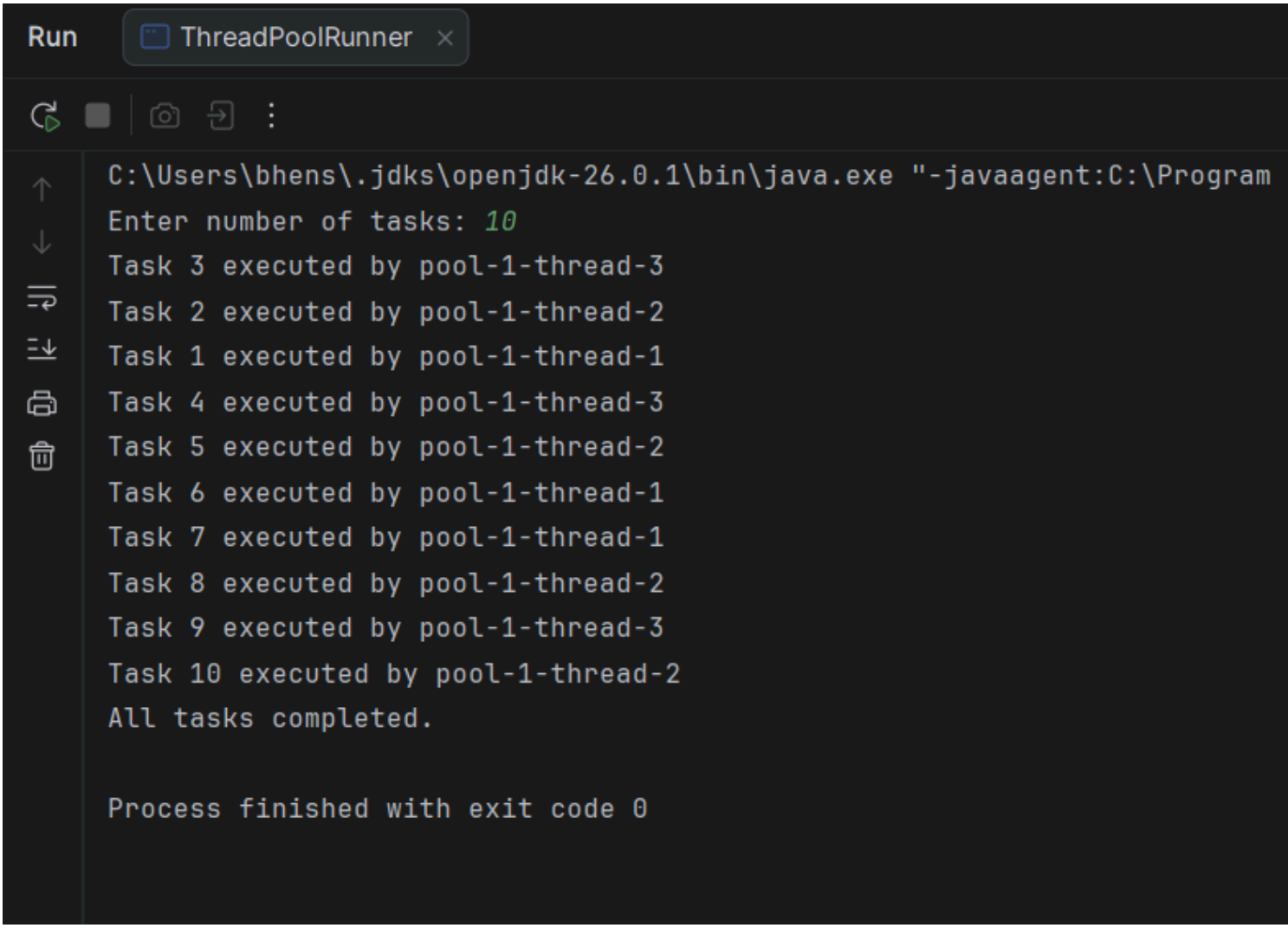
## Part A

**1. Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused.**

## CODE:



# OUTPUT:




**2. Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/ notify(); produce and consume a fixed number of items in order with none lost.**

# CODE:

```
import java.util.Scanner;

class Buffer {
   int value;
   boolean available = false;

   synchronized void produce(int n) throws InterruptedException {
       while (available) {
           wait();
       }

       value = n;
       available = true;
       System.out.println("Produced: " + n);
       notifyAll();
   }

   synchronized int consume() throws InterruptedException {
       while (!available) {
           wait();
       }

       int n = value;
       available = false;
       System.out.println("Consumed: " + n);
       notifyAll();
       return n;
   }
}

public class ProducerConsumer {
   public static void main(String[] args) throws InterruptedException {
       Scanner sc = new Scanner(System.in);

       System.out.print("Enter number of items: ");
       int n = sc.nextInt();

       Buffer b = new Buffer();

       Thread producer = new Thread(() -> {
           for (int i = 1; i <= n; i++) {
               try {
                   b.produce(i);
               } catch (InterruptedException e) {
                   Thread.currentThread().interrupt();
                   return;
               }
           }
       });

       Thread consumer = new Thread(() -> {
           for (int i = 1; i <= n; i++) {
               try {
                   b.consume();
               } catch (InterruptedException e) {
                   Thread.currentThread().interrupt();
```

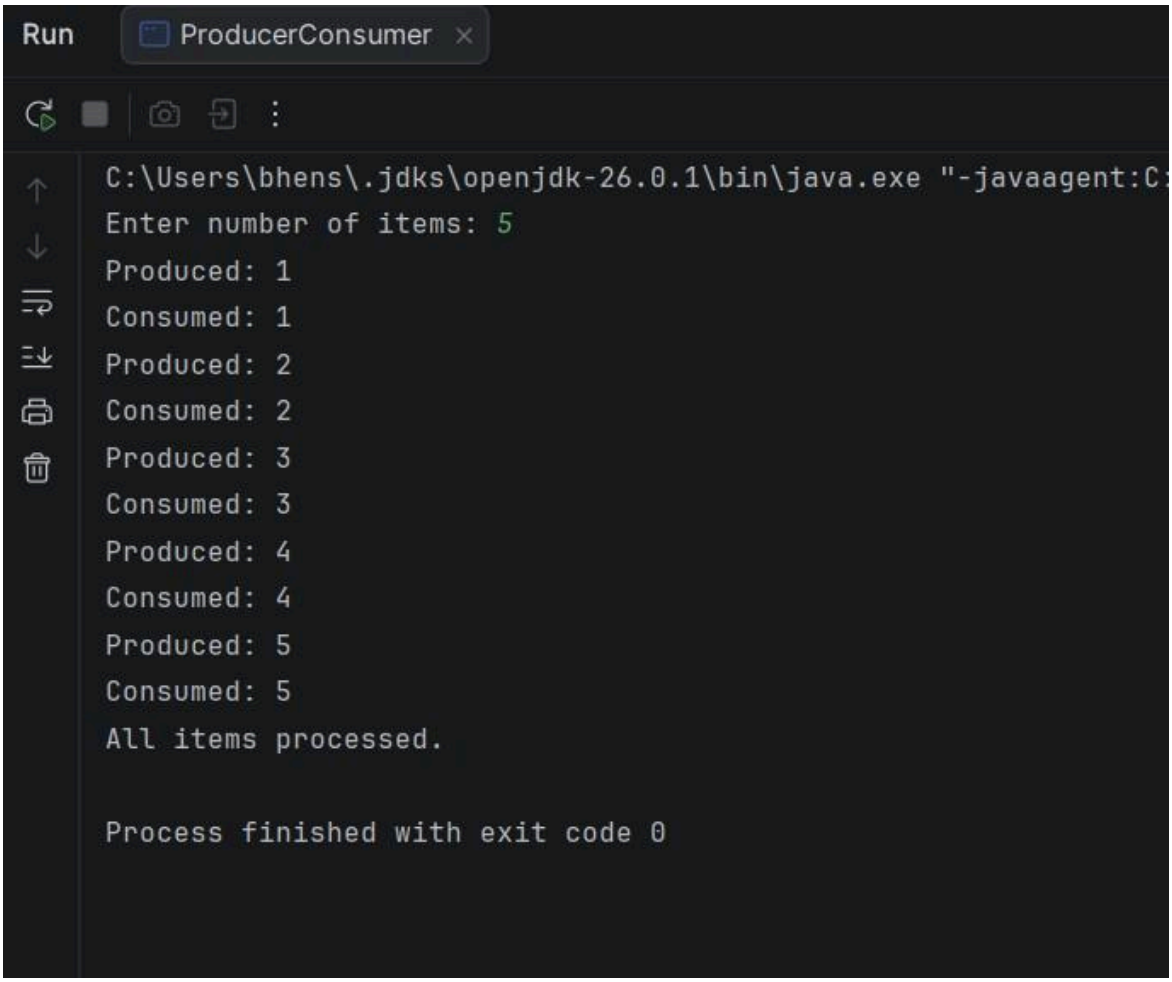
```
return;
                }
            }
        });

producer.start(); consumer.start();

producer.join(); consumer.join();

System.out.println("All items processed.");
        sc.close();
    }
}
```

# OUTPUT:




**3. Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself.**

# CODE:

```
import java.util.Scanner;

public class DeadlockDemo {

   static final Object lockA = new Object();
   static final Object lockB = new Object();

   static void deadlock() {
       Thread t1 = new Thread(() -> {
           synchronized (lockA) {
               System.out.println("Thread 1 locked A");

               synchronized (lockB) {
                   System.out.println("Thread 1 locked B");
               }
           }
       });

       Thread t2 = new Thread(() -> {
           synchronized (lockB) {
               System.out.println("Thread 2 locked B");

               synchronized (lockA) {
                   System.out.println("Thread 2 locked A");
               }
           }
       });

       t1.start();
       t2.start();
   }

   static void solve() {
       Thread t1 = new Thread(() -> {
           synchronized (lockA) {
               System.out.println("Thread 1 locked A");

               synchronized (lockB) {
                   System.out.println("Thread 1 locked B");
               }
           }
       });

       Thread t2 = new Thread(() -> {
           synchronized (lockA) {
               System.out.println("Thread 2 locked A");

               synchronized (lockB) {
                   System.out.println("Thread 2 locked B");
               }
           }
       });

       t1.start();
```

```
t2.start();

try {
t1.join();
t2.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

System.out.println("Deadlock avoided.");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

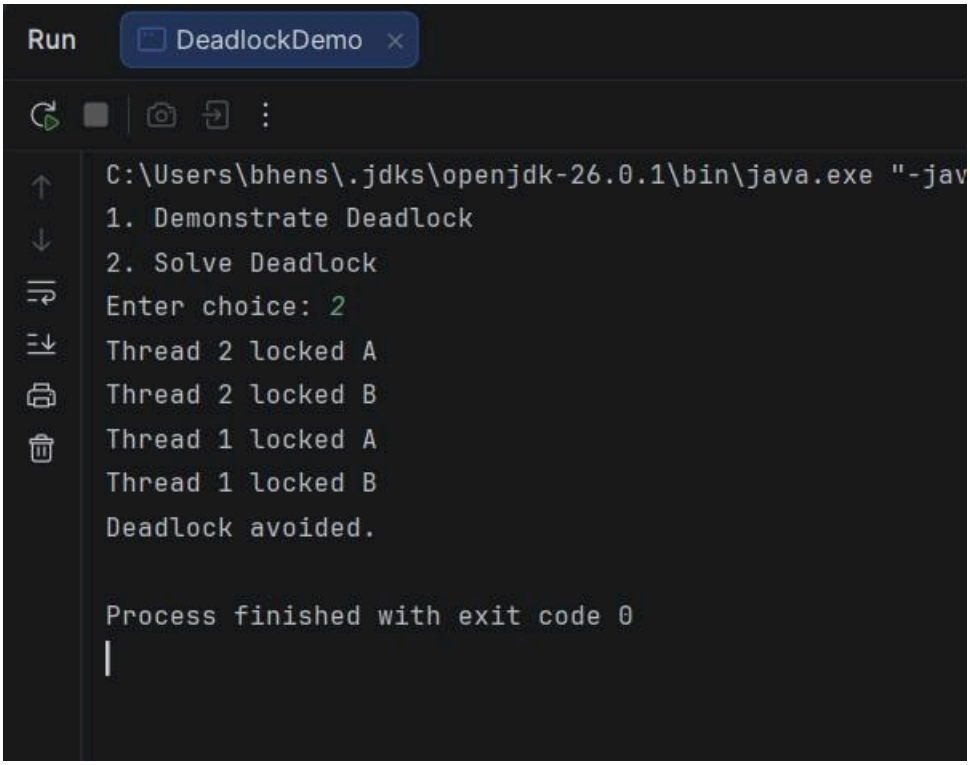
System.out.println("1. Demonstrate Deadlock"); System.out.println("2.
        Solve Deadlock"); System.out.print("Enter choice: ");

int choice = sc.nextInt();

if (choice == 1) {
            deadlock();
} else if (choice == 2) {
            solve();
} else {
System.out.println("Invalid choice.");
        }

sc.close();
    }
}
```

# OUTPUT:



# PRACTICAL 11

## 1. Save/load: make a small class Serializable, save an array of its objects to a file with

## ObjectOutputStream, then read it back into fresh objects and confirm the values survived.

## Mark one field transient and show it is not saved.

## CODE:

```
import java.io.*;
class Student implements Serializable {
    int id;
    String name;
    transient String password;
    Student(int id, String name, String password) {
        this.id = id;
        this.name = name;
        this.password = password;
    }
}
public class SerializationDemo {
    public static void main(String[] args) {
        try {
```

```
        Student[] students = {
            new Student(101, "Hardi", "abc123"),
            new Student(102, "Rahul", "xyz456")
        };
        ObjectOutputStream out = new ObjectOutputStream(
            new FileOutputStream("students.dat")
        );
        out.writeObject(students);
        out.close();

        System.out.println("Objects saved successfully.");
        ObjectInputStream in = new ObjectInputStream(
            new FileInputStream("students.dat")
        );

        Student[] loadedStudents = (Student[]) in.readObject();
        in.close();
        System.out.println("\nLoaded Student Details:");
        for (Student s : loadedStudents) {
            System.out.println("ID: " + s.id);
            System.out.println("Name: " + s.name);
            System.out.println("Password: " + s.password);
            System.out.println();
        }
```

```
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

## OUTPUT:

Objects saved successfully.

Loaded Student Details:

ID: 101

Name: Hardi

Password: null

ID: 102

Name: Rahul

Password: null

## 2. Log analyzer: create a few text files, then use Path/Files to read each line by line and

## count the total lines and how many contain a keyword across all files; also print each

## file’s size from its attributes.

## CODE:

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.List;

public class LogAnalyzer {
    public static void main(String[] args) {
        Path[] files = {
            Paths.get("log1.txt"),
            Paths.get("log2.txt"),
            Paths.get("log3.txt")
        };
        String keyword = "ERROR";
        int totalLines = 0;
        int matchingLines = 0;
        try {
            for (Path path : files) {
                if (Files.notExists(path)) {
                    if (path.endsWith("log1.txt")) {
                        Files.write(path, List.of(
                            "Application started",
                            "ERROR: File not found",
                            "User logged in"
                        ));
                    }
```

```
            else if (path.endsWith("log2.txt")) {
                Files.write(path, List.of(
                    "Server running",
                    "ERROR: Connection failed",
                    "ERROR: Timeout occurred"
                ));
            } else {
                Files.write(path, List.of(
                    "System running normally",
                    "User logged out"
                ));
            }
        }
        int fileLines = 0;
        int fileMatches = 0;
        for (String line : Files.readAllLines(path)) {
            fileLines++;
            if (line.contains(keyword)) {
                fileMatches++;
            }
        }
        totalLines += fileLines;
        matchingLines += fileMatches;
        BasicFileAttributes attr =
        Files.readAttributes(path, BasicFileAttributes.class);
```

```
                System.out.println("File: " + path.getFileName());
                System.out.println("Lines: " + fileLines);
                System.out.println("Matching lines: " + fileMatches);
                System.out.println("Size: " + attr.size() + " bytes");
                System.out.println();
            }
            System.out.println("Total lines: " + totalLines);
            System.out.println("Total lines containing "
                            + keyword + ": " + matchingLines);
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

## OUTPUT:

File: log1.txt

Lines: 3

Matching lines: 1

Size: ... bytes

File: log2.txt

Lines: 3

Matching lines: 2

Size: ... bytes

File: log3.txt

25IT006 : CEUE203

Lines: 2

Matching lines: 0

Size: ... bytes

Total lines: 8

Total lines containing ERROR: 3

## 3. (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of

## the files it contains (name, size, modified time), writing the summary to a file; decide

## what to include.

## CODE:

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.Instant;

public class FolderReport {
    public static void main(String[] args) {
        Path folder = Paths.get("myfolder");
        Path report = Paths.get("file_report.txt");
```

```
        try {
            StringBuilder result = new StringBuilder();
            result.append("FOLDER TREE FILE REPORT\n");
            result.append("======================\n\n");
            Files.walkFileTree(folder, new SimpleFileVisitor<Path>() {
                @Override
                public FileVisitResult visitFile(
                        Path file, BasicFileAttributes attrs)
                        throws IOException {
                    result.append("File Name: ")
                          .append(file.getFileName()).append("\n");
                    result.append("Full Path: ")
                          .append(file.toAbsolutePath()).append("\n");
                    result.append("Size: ")
                          .append(attrs.size()).append(" bytes\n");
                    result.append("Modified Time: ")
                          .append(Instant.ofEpochMilli(
                              attrs.lastModifiedTime().toMillis()))
                          .append("\n\n");
                    return FileVisitResult.CONTINUE;
                }
                @Override
                public FileVisitResult visitFileFailed(
                        Path file, IOException exc) {
                    result.append("Could not access: ")
```

```
                    .append(file).append("\n\n");
                return FileVisitResult.CONTINUE;
            }
        });
        Files.writeString(report, result.toString());
        System.out.println("Report generated successfully.");
        System.out.println("Saved to: " + report.toAbsolutePath());
    }
    catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
    }
  }
}
```

## OUTPUT:

FOLDER TREE FILE REPORT

File Name: notes.txt

Full Path: .../myfolder/notes.txt

Size: 120 bytes

Modified Time: 2026-10-28T07:30:00Z

File Name: report.txt

Full Path: .../myfolder/documents/report.txt

Size: 250 bytes

Modified Time: 2026-10-28T07:35:00Z

25IT006 : CEUE203

# PRACTICAL 12

## 1. Word frequency: split a paragraph into words, count each in a HashMap, and print the

## words sorted by frequency (highest first) using a Comparator.

## CODE:

```
import java.util.*;


public class WordFrequency {
    public static void main(String[] args) {
        String paragraph = "java is easy and java is powerful";
        String[] words = paragraph.toLowerCase().split("\\s+");
        HashMap<String, Integer> map = new HashMap<>();
        for (String word : words) {
            map.put(word, map.getOrDefault(word, 0) + 1);
        }
        List<Map.Entry<String, Integer>> list =
                new ArrayList<>(map.entrySet());
        list.sort(new Comparator<Map.Entry<String, Integer>>() {
            @Override
            public int compare(Map.Entry<String, Integer> a,
```

```
                        Map.Entry<String, Integer> b) {
                return b.getValue().compareTo(a.getValue());
            }
        });

        System.out.println("Word Frequencies:");
        for (Map.Entry<String, Integer> entry : list) {
            System.out.println(
                entry.getKey() + " : " + entry.getValue()
            );
        }
    }
}
```

## OUTPUT:

```
Word Frequencies:
java : 2
is : 2
powerful : 1
and : 1
easy : 1
```

## 2. Unique visitors: from a list of login names with repeats, use a HashSet to count unique

## users and a LinkedHashSet to keep first-seen order; print the unique count and the first

## three distinct names.

## CODE:

```
import java.util.*;


public class UniqueVisitors {
    public static void main(String[] args) {


        List<String> logins = Arrays.asList(
            "Hardi", "Rahul", "Priya", "Hardi",
            "Amit", "Rahul", "Neha", "Priya"
        );


        HashSet<String> uniqueUsers = new HashSet<>(logins);
        LinkedHashSet<String> orderedUsers =
                new LinkedHashSet<>(logins);
        System.out.println("Unique visitors: " + uniqueUsers.size());
        System.out.println("First three distinct visitors:");
        int count = 0;
        for (String name : orderedUsers) {
            if (count == 3) {
                break;
            }
            System.out.println(name);
```

```
            count++;
        }
    }
}
```

## OUTPUT:

Unique visitors: 5

First three distinct visitors:

Hardi

Yashvi

Diya

## 3. (For Advanced Learners – home) Build a leaderboard from player scores held in a Map

## and print the top-N sorted by score (ties broken by name); choose the data and sorting

## details.

## CODE:

```
import java.util.*;


public class Leaderboard {
    public static void main(String[] args) {
```

```
Map<String, Integer> scores = new HashMap<>();
scores.put("Rahul", 450);
scores.put("Hardi", 500);
scores.put("Priya", 450);
scores.put("Amit", 300);
scores.put("Neha", 500);
int N = 3;  // Display top 3 players

List<Map.Entry<String, Integer>> players =
        new ArrayList<>(scores.entrySet());

players.sort(new Comparator<Map.Entry<String, Integer>>() {
    @Override
    public int compare(Map.Entry<String, Integer> a,
                       Map.Entry<String, Integer> b) {

        int scoreCompare =
                b.getValue().compareTo(a.getValue());

        if (scoreCompare != 0) {
            return scoreCompare;
        }
        return a.getKey().compareTo(b.getKey());
    }
```

```
        });

        System.out.println("Top " + N + " Players:");
        for (int i = 0; i < Math.min(N, players.size()); i++) {
            Map.Entry<String, Integer> player = players.get(i);
            System.out.println(
                player.getKey() + " - " + player.getValue()
            );
        }
    }
}
```

## OUTPUT:

Top 3 Players:

Hardi - 500

Neha - 500

Priya – 450

# PRACTICAL 13

## 1. Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with

## different types to show type safety without casts, and write a generic method that returns

## the first element of any array.

## CODE:

```
class Box<T> {
    private T value;
    public void set(T value) {
        this.value = value;
    }
    public T get() {
        return value;
    }
}
class Pair<A, B> {
    private A first;
    private B second;
    Pair(A first, B second) {
        this.first = first;
        this.second = second;
```

```
    }
    public A getFirst() {
        return first;
    }
    public B getSecond() {
        return second;
    }
}
public class GenericDemo {
        public static <T> T firstElement(T[] array) {
        return array[0];
    }
    public static void main(String[] args)
    {
        Box<Integer> intBox = new Box<>();
        intBox.set(100);
        System.out.println("Integer Box: " + intBox.get());

        Box<String> strBox = new Box<>();
        strBox.set("Hello Java");
        System.out.println("String Box: " + strBox.get());

        Pair<String, Integer> student =
                new Pair<>("Hardi", 101);
        System.out.println("Name: " + student.getFirst());
```

```
        System.out.println("ID: " + student.getSecond());
        Integer[] numbers = {10, 20, 30};
        System.out.println(
            "First number: " + firstElement(numbers)
        );
        String[] names = {"Rahul", "Priya", "Neha"};
        System.out.println(
            "First name: " + firstElement(names)
        );
    }
}
```

## OUTPUT:

Integer Box: 100

String Box: Hello Java

Name: Hardi

ID: 101

First number: 10

First name: Rahul

## 2. Bounded stats: a generic method <N extends Number> double average(N[] values) that

## works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

# CODE:

```
public class BoundedStats {


    public static <N extends Number> double average(N[] values) {
        if (values.length == 0) {
            throw new IllegalArgumentException(
                "Array must not be empty"
            );
        }
        double sum = 0.0;
        for (N value : values) {
            sum += value.doubleValue();
        }
        return sum / values.length;
    }
    public static void main(String[] args) {
        Integer[] intValues = {10, 20, 30};
        Double[] doubleValues = {2.5, 3.5, 4.0};
        Long[] longValues = {100L, 200L, 300L};
        System.out.println(
            "Integer average: " + average(intValues)
        );
        System.out.println(
            "Double average: " + average(doubleValues)
        );
```

```
        System.out.println(
            "Long average: " + average(longValues)
        );
    }
}
```

## OUTPUT:

Integer average: 20.0

Double average: 3.3333333333333335

Long average: 200.0

## 3. (For Advanced Learners – home) Implement a small generic container of your choice

## (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty

## case sensibly.

## CODE:

```
class GenericStack<T> {
    private T[] stack;
    private int top;
    private int capacity;

    @SuppressWarnings("unchecked")
```

```
    public GenericStack(int capacity) {
        this.capacity = capacity;
        stack = (T[]) new Object[capacity];
        top = -1;
    }
    public void push(T value) {
        if (top == capacity - 1) {
            System.out.println("Stack is full!");
            return;
        }
        stack[++top] = value;
    }
    public T pop() {
        if (top == -1) {
            System.out.println("Stack is empty!");
            return null;
        }
        T value = stack[top];
        stack[top] = null;
        top--;


        return value;
    }
    public void display() {
        if (top == -1) {
```

```
            System.out.println("Stack is empty!");
            return;
        }
        for (int i = top; i >= 0; i--) {
            System.out.println(stack[i]);
        }
    }
}
public class GenericStackDemo {
    public static void main(String[] args) {

        GenericStack<Integer> intStack = new GenericStack<>(5);
        intStack.push(10);
        intStack.push(20);
        intStack.push(30);
        System.out.println("Integer Stack:");
        intStack.display();
        System.out.println("Popped: " + intStack.pop());
        GenericStack<String> strStack = new GenericStack<>(5);

        strStack.push("Java");
        strStack.push("Python");
        System.out.println("\nString Stack:");
        strStack.display();
        System.out.println("Popped: " + strStack.pop());
```

```
        strStack.pop();
        strStack.pop();
    }
}
```

## OUTPUT:

```
Integer Stack:
30
20
10
Popped: 30
String Stack:
Python
Java
Popped: Python
Stack is empty!
```

25IT006
CEUE203

# PRACTICAL 14

## 1. Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a

## menu main), then write two or three JUnit @Test methods checking that adding increases

## the count and an invalid action throws — practice before testing MiniBank.

## CODE:

File 1: Student.java — Model

```
public class Student {
    int id;
    String name;
    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

File 2: StudentService.java — Service

```
import java.util.ArrayList;

public class StudentService {
```

```
    private ArrayList<Student> students = new ArrayList<>();
    public void addStudent(Student s) {
        if (s == null) {
            throw new IllegalArgumentException("Invalid student");
        }
        students.add(s);
    }
    public int getCount() {
        return students.size();
    }
    public void removeStudent(int index) {
        if (index < 0 || index >= students.size()) {
            throw new IllegalArgumentException("Invalid index");
        }
        students.remove(index);
    }
}
```

File 3: Main.java — Menu-driven program

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StudentService service = new StudentService();
        while (true) {
```

```
        System.out.println("\n1. Add Student");
        System.out.println("2. Show Count");
        System.out.println("3. Exit");
        System.out.print("Enter choice: ");
        int choice = sc.nextInt();
        switch (choice) {
            case 1:
                System.out.print("Enter student ID: ");
                int id = sc.nextInt();
                sc.nextLine();
                System.out.print("Enter student name: ");
                String name = sc.nextLine();
                service.addStudent(new Student(id, name));
                System.out.println("Student added successfully!");
                break;
            case 2:
                System.out.println("Total students: "
                        + service.getCount());
                break;
            case 3:
                sc.close();
                return;
            default:
                System.out.println("Invalid choice!");
        }
```

```
        }
    }
}
```

File 4: StudentServiceTest.java — JUnit tests

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class StudentServiceTest {
    @Test
    void testAddingIncreasesCount() {
        StudentService service = new StudentService();
        assertEquals(0, service.getCount());
        service.addStudent(new Student(1, "Amit"));
        assertEquals(1, service.getCount());
    }
    @Test
    void testAddingNullThrowsException() {
        StudentService service = new StudentService();
        assertThrows(
            IllegalArgumentException.class,
            () -> service.addStudent(null)
        );
    }
    @Test
    void testInvalidIndexThrowsException() {
```

```
        StudentService service = new StudentService();
        assertThrows(
            IllegalArgumentException.class,
            () -> service.removeStudent(0)
        );
    }
}
```

## OUTPUT:

```
1. Add Student
2. Show Count
3. Exit
Enter choice: 1
Enter student ID: 1
Enter student name: Amit
Student added successfully!

1. Add Student
2. Show Count
3. Exit
Enter choice: 1
Enter student ID: 2
Enter student name: Riya
Student added successfully!

1. Add Student
```

2. Show Count

3. Exit

Enter choice: 2

Total students: 2

1. Add Student

2. Show Count

3. Exit

Enter choice: 3

□==========□

## 2. (For Advanced Learners – home) Organise that app into packages, build a runnable JAR,

## and add a test that saves and reloads its data unchanged.

## CODE:

1. Student.java

```
package model;

import java.io.Serializable;
import java.util.Objects;
public class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String name;
```

```
    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (!(obj instanceof Student)) {
            return false;
        }
        Student other = (Student) obj;
        return id == other.id &&
                Objects.equals(name, other.name);
    }
    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }
    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "'}";
    }
}
```

2. StudentService.java

```
package service;
import model.Student;
import java.io.*;
import java.util.ArrayList;

public class StudentService {
    private ArrayList<Student> students = new ArrayList<>();
    public void addStudent(Student student) {
        if (student == null) {
            throw new IllegalArgumentException(
                "Student cannot be null"
            );
        }
        students.add(student);
    }
    public int getCount() {
        return students.size();
    }
    public Student getStudent(int index) {
        if (index < 0 || index >= students.size()) {
            throw new IllegalArgumentException(
                "Invalid student index"
            );
        }
        return students.get(index);
    }
    public void saveData(String filename) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream(filename))) {
            out.writeObject(students);
        }
    }
    @SuppressWarnings("unchecked")
    public void loadData(String filename)
            throws IOException, ClassNotFoundException {
```

```
            try (ObjectInputStream in = new ObjectInputStream(
                    new FileInputStream(filename))) {
                Object data = in.readObject();

                if (!(data instanceof ArrayList<?>)) {
                    throw new IOException("Invalid student data");
                }
                ArrayList<?> loaded = (ArrayList<?>) data;
                for (Object item : loaded) {
                    if (!(item instanceof Student)) {
                        throw new IOException("Invalid student record");
                    }
                }
                students = (ArrayList<Student>) loaded;
            }
        }
    }
```

3. Main.java

```
package app;

import model.Student;
import service.StudentService;
import java.io.IOException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        StudentService service = new StudentService();
        Scanner sc = new Scanner(System.in);

        try {
            service.loadData("students.dat");
            System.out.println("Previous data loaded.");
```

```
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("No saved data found. Starting fresh.");
        }

        while (true) {
            System.out.println("\n--- Student Management System ---");
            System.out.println("1. Add Student");
            System.out.println("2. Show Student Count");
            System.out.println("3. Save Data");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");

            int choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter student ID: ");
                    int id = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Enter student name: ");
                    String name = sc.nextLine();

                    service.addStudent(new Student(id, name));
                    System.out.println("Student added successfully!");
                    break;

                case 2:
                    System.out.println(
                        "Total students: " + service.getCount()
                    );
                    break;

                case 3:
                    try {
```

```
                    service.saveData("students.dat");
                    System.out.println("Data saved successfully.");
                } catch (IOException e) {
                    System.out.println("Error saving data: "
                            + e.getMessage());
                }
                break;

            case 4:
                sc.close();
                System.out.println("Exiting application.");
                return;

            default:
                System.out.println("Invalid choice. Try again.");
        }
    }
}
```

4. StudentServiceTest.java

```
package service;

import model.Student;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

import java.io.File;
import java.io.IOException;

import static org.junit.jupiter.api.Assertions.*;

class StudentServiceTest {
```

```
    @TempDir
    File tempDir;

    @Test
    void testAddingIncreasesCount() {
        StudentService service = new StudentService();

        assertEquals(0, service.getCount());

        service.addStudent(new Student(1, "Amit"));

        assertEquals(1, service.getCount());

        service.addStudent(new Student(2, "Riya"));

        assertEquals(2, service.getCount());
    }

    @Test
    void testInvalidActionThrowsException() {
        StudentService service = new StudentService();

        assertThrows(
            IllegalArgumentException.class,
            () -> service.addStudent(null)
        );

        assertThrows(
            IllegalArgumentException.class,
            () -> service.getStudent(0)
        );
    }

    @Test
    void testSaveAndReloadData()
```

```
            throws IOException, ClassNotFoundException {

        StudentService original = new StudentService();

        original.addStudent(new Student(1, "Amit"));
        original.addStudent(new Student(2, "Riya"));

        File file = new File(tempDir, "students.dat");
        String path = file.getAbsolutePath();

        original.saveData(path);

        StudentService restored = new StudentService();
        restored.loadData(path);

        assertEquals(original.getCount(), restored.getCount());

        for (int i = 0; i < original.getCount(); i++) {
            assertEquals(
                original.getStudent(i),
                restored.getStudent(i)
            );
        }
    }
}
```

5. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
```

```
    <groupId>com.example</groupId>
    <artifactId>StudentApp</artifactId>
    <version>1.0</version>

    <properties>
      <maven.compiler.release>17</maven.compiler.release>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncodi
ng>
    </properties>

    <dependencies>
      <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.11.0</version>
        <scope>test</scope>
      </dependency>
    </dependencies>

    <build>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.5.2</version>
        </plugin>

        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-jar-plugin</artifactId>
          <version>3.4.2</version>
          <configuration>
            <archive>
              <manifest>
```

```
                        <mainClass>app.Main</mainClass>
                    </manifest>
                </archive>
            </configuration>
        </plugin>
    </plugins>
</build>
</project>
```

## OUTPUT:

No saved data found. Starting fresh.

--- Student Management System ---

1. Add Student

2. Show Student Count

3. Save Data

4. Exit

Enter choice: 1

Enter student ID: 1

Enter student name: Amit

Student added successfully!

--- Student Management System ---

1. Add Student

2. Show Student Count

3. Save Data

4. Exit

Enter choice: 3

Data saved successfully.

25IT006
CEUE203

□==========□